Arrays Problems — Complete Solutions

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

8. Largest Element in an Array

Understanding the Problem NEW

You're handed a list of numbers and asked: which one is biggest? There's no trick here — you just need to look at every number once and remember the biggest you've seen so far. Think of it like scanning a row of people's heights and keeping a mental note of the tallest person you've spotted as you walk down the line.

Brute Force Approach NEW

A beginner's first instinct is often to compare every element with every other element (nested loops) to confirm which one is bigger than all the rest — that's `O(n²)` work for something that doesn't need it.

Why We Optimize NEW

The insight: you don't need to compare every pair. You only need **one running variable** — the biggest value seen so far — and update it as you go. One pass, one comparison per element, done. That takes brute force's O(n²) down to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15]

          Step 1: max = 10
          [10, 25, 5, 30, 15]
           ↑
          max=10

          Step 2: Compare 25 > 10 → max = 25
          [10, 25, 5, 30, 15]
               ↑
             max=25

          Step 3: Compare 5 > 25 → No
          [10, 25, 5, 30, 15]
```

```
                  ↑
                max=25

          Step 4: Compare 30 > 25 → max = 30
          [10, 25, 5, 30, 15]
                       ↑
                    max=30

          Step 5: Compare 15 > 30 → No
          [10, 25, 5, 30, 15]
                           ↑

                        max=30

       Result: 30
```

Solution Approach

Concept: Initialize max with first element and traverse the array.

Algorithm

```
Algorithm FIND_MAX(array):
           max = array[0]
           For i = 1 to n-1:
               If array[i] > max:
                   max = array[i]
           Return max
```

Java Implementation

```
class Solution {
           public int findLargest(int[] arr) {
               int max = arr[0];
               for (int i = 1; i < arr.length; i++) {
                   if (arr[i] > max) {
                        max = arr[i];
                   }
               }
               return max;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

9. Second Largest Element in an Array

Understanding the Problem NEW

Now you want not just the biggest number, but the one right behind it — the second highest. If the array were `[10, 25, 5, 30, 15]`, the largest is 30 and the second largest is 25 (not 15, and not another copy of 30 if it repeated).

Brute Force Approach NEW

The obvious first approach: sort the whole array, then read off the second-to-last element. Sorting costs `O(n log n)` — way more work than this simple question deserves.

Why We Optimize NEW

Just like finding the max needed only one running variable, finding the top-2 needs only **two** running variables — `first` and `second` — updated together in a single pass. Whenever a new number beats `first`, the old `first` slides down into `second` before being replaced. This avoids sorting entirely and gets you to O(n).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the second largest element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15]

       Step 1: first = 10, second = -inf
       [10, 25, 5, 30, 15]
        ↑
       first=10, second=-∞

       Step 2: 25 > 10 → second=10, first=25
       [10, 25, 5, 30, 15]
            ↑
          first=25, second=10

       Step 3: 5 between second and first → No
```

```
[10, 25, 5, 30, 15]
         ↑

Step 4: 30 > 25 → second=25, first=30
[10, 25, 5, 30, 15]
             ↑
           first=30, second=25

Step 5: 15 between second and first → No
[10, 25, 5, 30, 15]
                 ↑

Result: 25
```

Solution Approach

Concept: Track both largest and second largest in one pass.

Algorithm

```
Algorithm FIND_SECOND_MAX(array):
           first = -∞, second = -∞
           For each element x in array:
               If x > first:
                   second = first
                   first = x
               Else If x > second and x != first:
                   second = x
           Return second
```

Java Implementation

```
class Solution {
           public int findSecondLargest(int[] arr) {
               int first = Integer.MIN_VALUE;
               int second = Integer.MIN_VALUE;

                 for (int num : arr) {
                     if (num > first) {
                         second = first;
                         first = num;
                     } else if (num > second && num != first) {

                          second = num;
                    }
              }
              return second;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

10. Check if the Array is Sorted

Understanding the Problem NEW

You're checking a simple yes/no question: as you read the array left to right, does every number stay the same or get bigger than the one before it? If even once a number is smaller than its predecessor, the array isn't sorted.

Brute Force Approach NEW

A naive approach some beginners try is comparing every pair of elements (i, j) to check i comes before j only if arr[i] <= arr[j] for all such pairs — unnecessary $O(n^2)$ work.

Why We Optimize NEW

Sortedness is really a statement about **neighbours**, not about every possible pair. If every adjacent pair is in order, the whole array must be in order (this is easy to prove — if a ≤ b and b ≤ c, then a ≤ c automatically). So checking only n-1 adjacent pairs is both correct and sufficient, giving O(n) instead of $O(n^2)$.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if an array is sorted in non-decreasing order.

Visual Diagram

```
Array: [1, 2, 3, 4, 5]

       Step 1: 1 <= 2 → Continue
       [1, 2, 3, 4, 5]
        ↑ ↑

       Step 2: 2 <= 3 → Continue
       [1, 2, 3, 4, 5]
           ↑ ↑

       Step 3: 3 <= 4 → Continue
       [1, 2, 3, 4, 5]
              ↑ ↑
```

```
Step 4: 4 <= 5 → Continue
[1, 2, 3, 4, 5]
          ↑ ↑

Result: Sorted

Array: [1, 3, 2, 4, 5]
Step: 3 > 2 → Not Sorted
[1, 3, 2, 4, 5]
    ↑ ↑
```

Solution Approach

Concept: Check if each element is less than or equal to next element.

Algorithm

```
Algorithm IS_SORTED(array):
          For i = 0 to n-2:
              If array[i] > array[i+1]:
                  Return false
          Return true
```

Java Implementation

```
class Solution {
          public boolean isSorted(int[] arr) {
              for (int i = 0; i < arr.length - 1; i++) {
                  if (arr[i] > arr[i+1]) {
                       return false;
                  }
              }
              return true;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

11. Remove Duplicates from Sorted Array

Understanding the Problem NEW

Given a sorted array with repeated values like `[1,1,2,2,3,4,4,5]`, you need to squeeze out the duplicates and shift the remaining unique values to the front, in-place (without allocating a new array), then report how many unique values remain.

Brute Force Approach NEW

A beginner might use a separate hash set or list to collect unique values, then copy them back into the original array. This works but uses O(n) extra space, when the sorted structure of the input makes that unnecessary.

Why We Optimize NEW

Because the array is **already sorted**, duplicates are always sitting right next to each other. That means you can use two pointers: one (`i`) marking the last confirmed-unique position, and one (`j`) scanning ahead. Whenever `arr[j]` differs from `arr[i]`, you've found a new unique value — advance `i` and write it there. This gets the job done in-place in O(n) time and O(1) extra space, exploiting the sorted order instead of ignoring it.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Remove duplicates from sorted array in-place.

Visual Diagram

```
Array: [1, 1, 2, 2, 3, 4, 4, 5]

       Step 1: i=0, j=1
       [1, 1, 2, 2, 3, 4, 4, 5]
        ↑ ↑
       i j

       Step 2: arr[0] == arr[1] → j++
       [1, 1, 2, 2, 3, 4, 4, 5]
         ↑     ↑
       i     j
```

```
Step 3: arr[1] != arr[2] → arr[++i] = arr[j]
[1, 2, 2, 2, 3, 4, 4, 5]
    ↑     ↑
    i     j

Step 4: Continue...
[1, 2, 3, 4, 5, 4, 4, 5]
          ↑       ↑
          i       j

Result: [1, 2, 3, 4, 5]
```

Solution Approach

Concept: Use two-pointer technique to overwrite duplicates.

Algorithm

```
Algorithm REMOVE_DUPLICATES(array):
          If array empty: Return 0
          i = 0
          For j = 1 to n-1:
              If array[j] != array[i]:
                  i++
                  array[i] = array[j]
          Return i+1
```

Java Implementation

```
class Solution {
          public int removeDuplicates(int[] arr) {
              if (arr.length == 0) return 0;

                int i = 0;
                for (int j = 1; j < arr.length; j++) {
                    if (arr[j] != arr[i]) {
                        i++;
                        arr[i] = arr[j];
                    }
                }
                return i + 1;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

12. Left Rotate an Array by One Place

Understanding the Problem NEW

You need to shift every element one position to the left, and whatever falls off the front wraps around to become the new last element. `[1,2,3,4,5]` becomes `[2,3,4,5,1]`.

Brute Force Approach NEW

Beginners sometimes build a brand-new array, copy elements 1..n-1 into positions 0..n-2, then place the old first element at the end — correct, but it uses O(n) extra space unnecessarily.

Why We Optimize NEW

You can do this in-place: save the first element in a temporary variable (so it isn't lost), shift everything else left by writing over it, then drop the saved value into the last slot. Same O(n) time, but O(1) space instead of O(n) — no separate array needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate array elements left by one position.

Visual Diagram

```
Array: [1, 2, 3, 4, 5]

       Step 1: Store first element
       temp = 1
       [1, 2, 3, 4, 5]

       Step 2: Shift left
       [2, 3, 4, 5, 5]

       Step 3: Place temp at end
       [2, 3, 4, 5, 1]

       Result: [2, 3, 4, 5, 1]
```

Solution Approach

Concept: Store first element, shift all elements left, place stored element at end.

Algorithm

```
Algorithm LEFT_ROTATE_BY_ONE(array):
           temp = array[0]
           For i = 0 to n-2:
               array[i] = array[i+1]
           array[n-1] = temp
           Return array
```

Java Implementation

```
class Solution {
           public void leftRotateByOne(int[] arr) {
               int temp = arr[0];
               for (int i = 0; i < arr.length - 1; i++) {
                   arr[i] = arr[i+1];
               }
               arr[arr.length - 1] = temp;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

13. Left Rotate an Array by D Places

Understanding the Problem NEW

Same idea as before, but now you rotate left by `D` places instead of just one — e.g. rotating `[1,2,3,4,5,6,7]` left by 3 gives `[4,5,6,7,1,2,3]`.

Brute Force Approach NEW

The straightforward brute idea: rotate by one place, D separate times, in a loop. Each single rotation costs O(n), so doing it D times costs O(n·D) — slow when D is large.

Why We Optimize NEW

The elegant trick is the **reversal algorithm**: reverse the first D elements, reverse the remaining n-D elements, then reverse the whole array. Each reversal is O(n), and you only do 3 of them regardless of D, giving a flat O(n) total instead of O(n·D). This is a good example of how a clever transformation (reversing pieces) can replace repeated brute work.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate array elements left by D positions.

Visual Diagram

```
Array: [1, 2, 3, 4, 5, 6, 7], D = 3

       Step 1: Reverse first D elements
       [3, 2, 1, 4, 5, 6, 7]

       Step 2: Reverse remaining elements
       [3, 2, 1, 7, 6, 5, 4]

       Step 3: Reverse whole array
       [4, 5, 6, 7, 1, 2, 3]

       Result: [4, 5, 6, 7, 1, 2, 3]
```

Solution Approach

Concept: Use reversal algorithm (three reverses).

Algorithm

```
Algorithm LEFT_ROTATE_D(array, d):
           d = d % n
           REVERSE(array, 0, d-1)
           REVERSE(array, d, n-1)
           REVERSE(array, 0, n-1)
           Return array
```

Java Implementation

```
class Solution {
           public void leftRotate(int[] arr, int d) {
               int n = arr.length;
               d = d % n;
               reverse(arr, 0, d-1);
               reverse(arr, d, n-1);
               reverse(arr, 0, n-1);
           }

           private void reverse(int[] arr, int start, int end) {
               while (start < end) {
                   int temp = arr[start];
                   arr[start] = arr[end];
                   arr[end] = temp;
                   start++;
                   end--;
               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

14. Move Zeros to End

Understanding the Problem NEW

Given an array with some zeros scattered in it, like `[0,1,0,3,12]`, push all the zeros to the end while keeping the relative order of the non-zero numbers the same, producing `[1,3,12,0,0]`.

Brute Force Approach NEW

A beginner approach: create a new array, copy all non-zero elements into it first, then fill the rest with zeros, and copy back. This works but needs O(n) extra space.

Why We Optimize NEW

You can do it in-place with two pointers: one pointer marks where the next non-zero element should go; you scan through the array, and every time you see a non-zero value you swap it into that position and advance the marker. By the end, all non-zero values have been shifted left in order, and the zeros naturally end up at the back — all in O(n) time and O(1) space.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Move all zeros to the end while maintaining relative order of non-zero elements.

Visual Diagram

```
Array: [0, 1, 0, 3, 12]

       Step 1: i=0, j=0
       [0, 1, 0, 3, 12]
        ↑
       i,j

       Step 2: arr[j]==0 → j++
       [0, 1, 0, 3, 12]
          ↑  ↑
       i    j

       Step 3: arr[j]!=0 → swap arr[i], arr[j]
```

```
       [1, 0, 0, 3, 12]
           ↑ ↑
           i j

       Step 4: i++, j++
       [1, 0, 0, 3, 12]
              ↑ ↑
              i j

       Step 5: arr[j]!=0 → swap arr[i], arr[j]
       [1, 3, 0, 0, 12]
                 ↑ ↑
                 i j

       Step 6: i++, j++
       [1, 3, 0, 0, 12]
                    ↑ ↑
                    i j

       Step 7: arr[j]!=0 → swap arr[i], arr[j]
       [1, 3, 12, 0, 0]
                     ↑ ↑
                     i j

       Result: [1, 3, 12, 0, 0]
```

Solution Approach

Concept: Two-pointer technique - one pointer for non-zero elements.

Algorithm

```
Algorithm MOVE_ZEROS(array):
          i = 0
          For j = 0 to n-1:
              If array[j] != 0:
                  Swap array[i] and array[j]
                  i++
          Return array
```

Java Implementation

```
class Solution {
         public void moveZeroes(int[] arr) {
             int i = 0;
             for (int j = 0; j < arr.length; j++) {
                 if (arr[j] != 0) {
                      int temp = arr[i];
                      arr[i] = arr[j];
                      arr[j] = temp;
```

```
                    i++;
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

15. Linear Search

Understanding the Problem NEW

You're given an array and a target value, and you need to find the index of that value in the array (or report that it isn't there). There's no sorted order assumed here.

Brute Force Approach NEW

This *is* the brute-force-friendly problem: without any extra structure (sorting, hashing), the only correct way to guarantee finding a value in an unsorted array is to check every element one at a time.

Why We Optimize NEW

Linear Search is a case where the 'brute force' approach is actually already optimal — there's no way to search an arbitrary unsorted array faster than O(n) in the worst case, because the target could be anywhere, including the very last position. The lesson here is important: **optimization is only possible when there's exploitable structure** (sorted data → binary search, repeated lookups → hashing). Plain unsorted, one-shot search has no shortcut.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the index of a target element in an array.

Visual Diagram

```
Array: [10, 25, 5, 30, 15], Target: 30

       Step 1: Compare arr[0]=10 == 30? No
       [10, 25, 5, 30, 15]
        ↑

       Step 2: Compare arr[1]=25 == 30? No
       [10, 25, 5, 30, 15]
            ↑

       Step 3: Compare arr[2]=5 == 30? No
       [10, 25, 5, 30, 15]
```

```
                ↑

       Step 4: Compare arr[3]=30 == 30? Yes!
       [10, 25, 5, 30, 15]
                    ↑

       Result: Index = 3
```

Solution Approach

Concept: Traverse array and compare each element with target.

Algorithm

```
Algorithm LINEAR_SEARCH(array, target):
           For i = 0 to n-1:
               If array[i] == target:
                   Return i
           Return -1
```

Java Implementation

```
class Solution {
           public int linearSearch(int[] arr, int target) {
               for (int i = 0; i < arr.length; i++) {
                   if (arr[i] == target) {
                        return i;
                   }
               }
               return -1;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

16. Find the Union

Understanding the Problem NEW

You're given two *sorted* arrays and need to produce a single sorted array containing every value that appears in either one, with no duplicates — like combining two sorted guest lists into one, without repeating a name.

Brute Force Approach NEW

A naive approach: dump both arrays into a hash set to remove duplicates, then sort the result. That costs O((n+m) log(n+m)) for the final sort, ignoring the fact both inputs were already sorted.

Why We Optimize NEW

Because both arrays are sorted, you can walk them simultaneously with **two pointers**, always advancing whichever pointer has the smaller current value (and skipping over duplicates as you go). This produces the merged, duplicate-free result in a single O(n+m) pass with no extra sorting needed — reusing the sorted order you were already given instead of throwing it away.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find union of two sorted arrays.

Visual Diagram

```
Array1: [1, 2, 3, 4, 5]
       Array2: [2, 3, 5, 6, 7]

       Step-by-step:
       i=0, j=0: 1<2 → add 1, i++
       i=1, j=0: 2==2 → add 2, i++, j++
       i=2, j=1: 3==3 → add 3, i++, j++
       i=3, j=2: 4<5 → add 4, i++
       i=4, j=2: 5==5 → add 5, i++, j++
       i=5, j=3: j<len → add 6, j++
       i=5, j=4: j<len → add 7, j++
```

```
       Result: [1, 2, 3, 4, 5, 6, 7]
```

Solution Approach

Concept: Use two-pointer technique for sorted arrays.

Algorithm

```
Algorithm UNION(arr1, arr2):
           i=0, j=0
           result = []
           While i < n and j < m:
               If arr1[i] < arr2[j]:
                    Add arr1[i] to result if not duplicate
                    i++
               Else If arr1[i] > arr2[j]:
                    Add arr2[j] to result if not duplicate
                    j++
               Else:
                    Add arr1[i] to result
                    i++; j++
           Add remaining elements
           Return result
```

Java Implementation

```
class Solution {
           public List<Integer> union(int[] arr1, int[] arr2) {
               List<Integer> result = new ArrayList<>();
               int i = 0, j = 0;

                 while (i < arr1.length && j < arr2.length) {
                     if (arr1[i] < arr2[j]) {
                         if (result.isEmpty() || result.get(result.size()-1) !=
arr1[i]) {
                              result.add(arr1[i]);
                         }
                         i++;
                     } else if (arr1[i] > arr2[j]) {
                         if (result.isEmpty() || result.get(result.size()-1) !=
arr2[j]) {
                              result.add(arr2[j]);
                         }
                         j++;
                     } else {

                      if (result.isEmpty() || result.get(result.size()-1) !=
arr1[i]) {
                          result.add(arr1[i]);
```

```
                    }
                    i++; j++;
                }
            }

            while (i < arr1.length) {
                if (result.isEmpty() || result.get(result.size()-1) != arr1[i]) {
                     result.add(arr1[i]);
                }
                i++;
            }

            while (j < arr2.length) {
                if (result.isEmpty() || result.get(result.size()-1) != arr2[j]) {
                     result.add(arr2[j]);
                }
                j++;
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n+m) | Space Complexity: O(n+m)

17. Find Missing Number in an Array

Understanding the Problem NEW

You're given an array that's supposed to contain every number from 0 to n exactly once, but one number is missing. Find which one. E.g. `[3,0,1]` is missing 2.

Brute Force Approach NEW

The obvious brute approach: for every number from 0 to n, scan the whole array checking if it's present. That's O(n) numbers × O(n) scan each = O(n²) total.

Why We Optimize NEW

There's a much cleverer shortcut: you know exactly what the sum of 0..n *should* be (using the formula n(n+1)/2). Subtract the *actual* sum of the array from that expected sum, and whatever's left over is the missing number. (XOR-ing all numbers 0..n together with all array elements works the same way and avoids overflow concerns.) This turns an O(n²) search into a single O(n) pass with basic arithmetic.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the missing number from 0 to n in an array of size n.

Visual Diagram

```
Array: [3, 0, 1, 4], n=4

       Method 1: Sum
       Expected sum = 0+1+2+3+4 = 10
       Actual sum = 3+0+1+4 = 8
       Missing = 10-8 = 2

       Method 2: XOR
       XOR all numbers: 3^0^1^4 = 6
       XOR all indices: 0^1^2^3^4 = 4
       Missing = 6^4 = 2
```

```
Result: 2
```

Solution Approach

Concept: Use sum formula or XOR.

Algorithm (Sum Method)

Algorithm MISSING_NUMBER(array, n):

expected_sum = n * (n+1) / 2

actual_sum = sum(array)

Return expected_sum - actual_sum

Algorithm (XOR Method)

Algorithm MISSING_NUMBER_XOR(array, n):

xor_all = 0

For i = 0 to n:

xor_all ^= i

For each num in array:

xor_all ^= num

Return xor_all

Java Implementation

```
class Solution {
           // Sum Method
           public int missingNumber(int[] arr) {
               int n = arr.length;
               int expectedSum = n * (n + 1) / 2;
               int actualSum = 0;
               for (int num : arr) {
                   actualSum += num;
               }
               return expectedSum - actualSum;
           }

           // XOR Method (prevents overflow)
           public int missingNumberXOR(int[] arr) {
               int xor = 0;
               for (int i = 0; i <= arr.length; i++) {
```

```
                xor ^= i;
            }

            for (int num : arr) {
                xor ^= num;
            }
            return xor;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

18. Maximum Consecutive Ones

Understanding the Problem NEW

Given a binary array (only 0s and 1s), find the longest unbroken run of consecutive 1s. E.g. in `[1,1,0,1,1,1]` the answer is 3 (the last three 1s).

Brute Force Approach NEW

A brute way: for every starting index, keep extending forward counting 1s until you hit a 0, tracking the best length found. That's checking every possible subarray start, costing $O(n^2)$ in the worst case.

Why We Optimize NEW

You don't need to restart from every index — you can track a **running counter** that increases by 1 for each consecutive 1, and resets to 0 the moment you see a 0, updating a 'best so far' as you go. One single left-to-right pass, O(n) time, no nested loops needed.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum number of consecutive 1s in a binary array.

Visual Diagram

```
Array: [1, 1, 0, 1, 1, 1, 0, 1, 1]

       Step 1: count=1, max=1
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
        ↑

       Step 2: count=2, max=2
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
           ↑

       Step 3: 0 → reset count=0
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
              ↑

       Step 4: count=1
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
```

```
                ↑

       Step 5: count=2
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
                   ↑

       Step 6: count=3, max=3
       [1, 1, 0, 1, 1, 1, 0, 1, 1]
                      ↑

       Step 7: 0 → reset
       Step 8-9: count=2

       Result: 3
```

Solution Approach

Concept: Count consecutive ones and track maximum.

Algorithm

```
Algorithm MAX_CONSECUTIVE_ONES(array):
           max_count = 0, count = 0
           For each element x in array:
               If x == 1:
                   count++
                   max_count = max(max_count, count)
               Else:
                   count = 0
           Return max_count
```

Java Implementation

```
class Solution {
           public int findMaxConsecutiveOnes(int[] arr) {
               int maxCount = 0;
               int count = 0;

                 for (int num : arr) {

                  if (num == 1) {
                      count++;
                      maxCount = Math.max(maxCount, count);
                  } else {
                      count = 0;
                  }
              }
              return maxCount;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

19. Find the Number that Appears Once

Understanding the Problem NEW

Every number in the array appears exactly twice, except one number that appears only once — find that one number. E.g. in `[4,1,2,1,2]` the answer is 4.

Brute Force Approach NEW

A beginner approach: for each element, count how many times it appears elsewhere in the array (nested loop), and return the one whose count is 1. That's $O(n^2)$. A slightly better brute idea uses a hashmap to count frequencies in O(n) time but O(n) extra space.

Why We Optimize NEW

There's a beautifully cheap trick using the **XOR** operation: XOR-ing a number with itself gives 0 (`a ^ a = 0`), and XOR-ing with 0 leaves a number unchanged (`a ^ 0 = a`). If you XOR every element in the array together, every pair cancels itself out to 0, leaving only the lone number standing. This gets you O(n) time *and* O(1) space — better than even the hashmap version — purely by exploiting a mathematical property of XOR.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the element that appears once in an array where every other element appears twice.

Visual Diagram

```
Array: [4, 1, 2, 1, 2]

       Step 1: XOR all elements
       4 ^ 1 = 5
       5 ^ 2 = 7
       7 ^ 1 = 6
       6 ^ 2 = 4

       XOR Properties:
       a ^ a = 0
       a ^ 0 = a
```

```
       4 ^ 1 ^ 2 ^ 1 ^ 2 = 4 ^ (1^1) ^ (2^2) = 4 ^ 0 ^ 0 = 4

       Result: 4
```

Solution Approach

Concept: Use XOR property where a^a=0.

Algorithm

```
Algorithm SINGLE_NUMBER(array):
           result = 0
           For each num in array:
               result ^= num
           Return result
```

Java Implementation

```
class Solution {
           public int singleNumber(int[] arr) {
               int result = 0;
               for (int num : arr) {
                   result ^= num;
               }
               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

20. Longest Subarray with Given Sum K (Positive Numbers)

Understanding the Problem NEW

Given an array of **positive** numbers and a target sum K, find the length of the longest contiguous subarray whose elements add up to exactly K.

Brute Force Approach NEW

The naive approach checks every possible subarray (every pair of start and end indices), summing each one and comparing to K — $O(n^2)$ or, if you recompute the sum from scratch each time, even $O(n^3)$.

Why We Optimize NEW

Because all numbers are **positive**, growing the window (adding an element) only ever increases the sum, and shrinking it only ever decreases the sum — this monotonic behaviour is exactly what a **sliding window** needs. You expand the right edge of a window, and whenever the sum exceeds K you shrink from the left, tracking the best length seen. Each pointer only moves forward, so the whole process is O(n) — the positivity of the numbers is precisely the property that makes this optimization valid (it wouldn't work with negative numbers, which is why problem 21 needs a different technique).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest subarray with sum K (positive numbers only).

Visual Diagram

```
Array: [1, 2, 3, 1, 1, 1], K=3

       Brute Force:
       Subarray [1,2] sum=3 length=2
       Subarray [3] sum=3 length=1
       Subarray [1,1,1] sum=3 length=3

       Optimal (Two Pointer):
       [1, 2, 3, 1, 1, 1]
```

```
        ↑
       left, right
       sum=1

       [1, 2, 3, 1, 1, 1]
        ↑ ↑
       left right
       sum=3 length=2

       [1, 2, 3, 1, 1, 1]
        ↑     ↑
       left   right
       sum=6 > 3 → left++

       [1, 2, 3, 1, 1, 1]
           ↑ ↑
          left right
       sum=5 > 3 → left++

       [1, 2, 3, 1, 1, 1]
              ↑ ↑
             left right
       sum=3 length=1

       Continue...
       [1, 2, 3, 1, 1, 1]
                 ↑ ↑
                left right
       sum=2

       [1, 2, 3, 1, 1, 1]
                 ↑    ↑
                left right
       sum=3 length=3

       Result: 3
```

Solution Approach

Concept: Two-pointer sliding window for positive numbers.

Algorithm

```
Algorithm LONGEST_SUBARRAY_SUM_K(arr, K):
           left=0, right=0, sum=0, max_len=0
           While right < n:

              sum += arr[right]
              While sum > K and left <= right:
                  sum -= arr[left]
                  left++
```

```
            If sum == K:
                max_len = max(max_len, right-left+1)
            right++
        Return max_len
```

Java Implementation

```
class Solution {
        public int longestSubarrayWithSumK(int[] arr, int K) {
            int left = 0, right = 0;
            int sum = 0, maxLen = 0;

            while (right < arr.length) {
                sum += arr[right];

                while (sum > K && left <= right) {
                    sum -= arr[left];
                    left++;
                }

                if (sum == K) {
                    maxLen = Math.max(maxLen, right - left + 1);
                }

                right++;
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

21. Longest Subarray with Sum K (Positive and Negative)

Understanding the Problem NEW

Same question as above, but now the array can contain **negative** numbers too, which breaks the sliding-window trick from problem 20 (a window's sum is no longer guaranteed to grow as it expands).

Brute Force Approach NEW

Checking every subarray directly is still O(n²) and doesn't get any easier just because negatives are allowed.

Why We Optimize NEW

The fix is to track **prefix sums**: the running total from the start of the array up to each index. If two prefix sums differ by exactly K, the subarray between those two points sums to K. Store the *first* index where each prefix sum was seen in a hashmap; then for each new prefix sum `S`, check if `S - K` has been seen before, and if so, compute the subarray length. This turns the problem into one hashmap lookup per element — O(n) time (with O(n) space for the map) — and works correctly even with negative numbers, unlike the sliding-window approach.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest subarray with sum K (including negative numbers).

Visual Diagram

```
Array: [1, 2, -1, 2, 3], K=4

       Prefix Sum Method:
       Index:   0 1 2 3      4
       Array:   1 2 -1 2     3
       Prefix: 1 3 2 4       7

       For target K=4:
       Check prefix[i] - K exists in map
```

```
       i=3: prefix=4, 4-4=0 exists? No
       But we can check: prefix[3] - prefix[0] = 4-1=3 (not K)
       prefix[3] - prefix[2] = 4-2=2 (not K)
       prefix[3] - prefix[1] = 4-3=1 (not K)

       Actually: subarray [1,2,-1,2] sum=4 length=4
       Found: prefix[3] - prefix[-1] (where we store 0 at index -1)

       Result: 4
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm LONGEST_SUBARRAY_K(arr, K):
           map = {0: -1}
           sum = 0, max_len = 0
           For i = 0 to n-1:
               sum += arr[i]
               If map contains (sum - K):
                   max_len = max(max_len, i - map[sum-K])
               If map doesn't contain sum:
                   map[sum] = i
           Return max_len
```

Java Implementation

```
class Solution {
           public int longestSubarrayWithSumK(int[] arr, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, -1);

                 int sum = 0, maxLen = 0;

                 for (int i = 0; i < arr.length; i++) {
                     sum += arr[i];

                    if (map.containsKey(sum - K)) {
                        maxLen = Math.max(maxLen, i - map.get(sum - K));
                    }

                    if (!map.containsKey(sum)) {
                        map.put(sum, i);

                 }
             }

             return maxLen;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

22. Two Sum Problem

Understanding the Problem NEW

You're given an array and a target sum, and need to find the indices of the two numbers that add up to exactly that target. E.g. `[2,7,11,15]`, target 9 → indices [0,1] since 2+7=9.

Brute Force Approach NEW

The obvious approach: for every pair of elements, check if they add up to the target — two nested loops, $O(n^2)$ comparisons.

Why We Optimize NEW

For each number `x` you visit, what you actually need to know is: 'have I already seen the number `target - x`?' Instead of searching the whole array again to answer that (which is what the nested-loop approach effectively does), you can store every number you've seen so far in a **hashmap** as you go, so that lookup becomes instant (O(1) average). This drops the total time to a single O(n) pass, at the cost of O(n) extra space for the map.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find two numbers that sum to target.

Visual Diagram

```
Array: [2, 7, 11, 15], Target: 9

       Brute Force:
       2+7=9 ✓ (0,1)
       2+11=13 ✗
       2+15=17 ✗
       7+11=18 ✗
       7+15=22 ✗
       11+15=26 ✗

       HashMap Method:
       Step 1: 2 → need=7 (store 2 at index 0)
       Step 2: 7 → need=2 (found at index 0)
```

```
       Return [0, 1]

       Result: [0, 1]
```

Solution Approach

Concept: Hashmap to store complement.

Algorithm

```
Algorithm TWO_SUM(arr, target):
           map = {}
           For i = 0 to n-1:
               complement = target - arr[i]
               If complement in map:
                    Return [map[complement], i]
               map[arr[i]] = i
           Return []
```

Java Implementation

```
class Solution {
           public int[] twoSum(int[] nums, int target) {
               Map<Integer, Integer> map = new HashMap<>();

                 for (int i = 0; i < nums.length; i++) {
                     int complement = target - nums[i];
                     if (map.containsKey(complement)) {
                         return new int[]{map.get(complement), i};
                     }
                     map.put(nums[i], i);
                 }
                 return new int[0];
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

23. Sort an Array of 0's, 1's and 2's

Understanding the Problem NEW

An array contains only the values 0, 1, and 2 in random order, and you need to sort it — but the challenge is to do it without calling a generic sorting algorithm, ideally in a single pass.

Brute Force Approach NEW

A beginner's first idea is to just run any comparison sort (like the ones from Chapter 1) on it — correct, but O(n log n), which ignores the fact there are only 3 distinct values, so much less work should be needed. A slightly better brute idea: count the 0s, 1s, and 2s, then overwrite the array in three passes — this works in O(n) but needs two full passes.

Why We Optimize NEW

The **Dutch National Flag** algorithm gets it done in a *single* pass using three pointers — `low`, `mid`, and `high` — that partition the array into 'confirmed 0s', 'confirmed 1s' (being scanned), and 'confirmed 2s' regions as you go, swapping elements into the right region the moment you see them. No sorting comparisons at all — just constant-time swaps based on which of the 3 fixed values you're looking at, giving O(n) time and O(1) space in one pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort array containing only 0, 1, 2 without using sorting algorithm.

Visual Diagram

```
Array: [2, 0, 2, 1, 1, 0]

       Dutch National Flag Algorithm:
       low=0, mid=0, high=5

       Step 1: arr[mid]=2 → swap with high
       [0, 0, 2, 1, 1, 2] high=4
            ↑
            mid
```

```
       Step 2: arr[mid]=0 → swap with low
       [0, 0, 2, 1, 1, 2] low=1, mid=1
               ↑
               mid

       Step 3: arr[mid]=0 → swap with low
       [0, 0, 2, 1, 1, 2] low=2, mid=2
                  ↑
                  mid

       Step 4: arr[mid]=2 → swap with high
       [0, 0, 1, 1, 2, 2] high=3
                  ↑
                  mid

       Step 5: arr[mid]=1 → mid++
       [0, 0, 1, 1, 2, 2] mid=4
                     ↑
                     mid

       Step 6: arr[mid]=1 → mid++
       [0, 0, 1, 1, 2, 2] mid=5

       Result: [0, 0, 1, 1, 2, 2]
```

Solution Approach

Concept: Dutch National Flag algorithm (3-pointer).

Algorithm

```
Algorithm SORT_012(arr):
          low=0, mid=0, high=n-1
          While mid <= high:
              If arr[mid] == 0:
                  Swap arr[low] and arr[mid]
                  low++; mid++
              Else If arr[mid] == 1:
                  mid++
              Else: // arr[mid] == 2
                  Swap arr[mid] and arr[high]
                  high--
          Return arr
```

Java Implementation

```
class Solution {
         public void sortColors(int[] nums) {
             int low = 0, mid = 0, high = nums.length - 1;
```

```
            while (mid <= high) {
                if (nums[mid] == 0) {
                    swap(nums, low, mid);
                    low++;
                    mid++;
                } else if (nums[mid] == 1) {
                    mid++;
                } else {
                    swap(nums, mid, high);
                    high--;
                }
            }
        }

        private void swap(int[] arr, int i, int j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

24. Majority Element (>n/2 times)

Understanding the Problem NEW

Find the element that appears **more than n/2 times** in the array (it's guaranteed such an element exists). E.g. in `[2,2,1,1,1,2,2]`, 2 appears more than half the time.

Brute Force Approach NEW

Beginners often count occurrences of every distinct element (nested loop or hashmap) and check which count exceeds n/2 — this works in O(n) with a hashmap, but uses O(n) extra space.

Why We Optimize NEW

Since the majority element appears more than *half* the array, it's guaranteed to 'outlast' every other element combined if you pair them off and cancel them out. That's the idea behind **Moore's Voting Algorithm**: keep a running candidate and a counter; increment the counter when you see the candidate again, decrement it for anything else, and switch candidates when the counter hits zero. Whatever candidate survives to the end must be the majority element. This achieves O(n) time with just O(1) space — no hashmap needed — because the 'more than half' guarantee is exactly what makes the cancellation trick mathematically safe.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find element that appears more than n/2 times.

Visual Diagram

```
Array: [2, 2, 1, 1, 1, 2, 2]

       Moore's Voting Algorithm:
       Step 1: candidate=2, count=1
       [2, 2, 1, 1, 1, 2, 2]
        ↑

       Step 2: 2==candidate → count=2
```

```
       [2, 2, 1, 1, 1, 2, 2]
           ↑

       Step 3: 1!=candidate → count=1
       [2, 2, 1, 1, 1, 2, 2]
              ↑

       Step 4: 1!=candidate → count=0
       [2, 2, 1, 1, 1, 2, 2]
                 ↑

       Step 5: count=0 → candidate=1, count=1
       [2, 2, 1, 1, 1, 2, 2]
                    ↑

       Step 6: 2!=candidate → count=0
       [2, 2, 1, 1, 1, 2, 2]
                       ↑

       Step 7: count=0 → candidate=2, count=1
       [2, 2, 1, 1, 1, 2, 2]
                          ↑

       Result: candidate=2 (verify: 2 appears 3 times > 3.5)
```

Solution Approach

Concept: Moore's Voting Algorithm.

Algorithm

```
Algorithm MAJORITY_ELEMENT(arr):
           candidate = -1, count = 0
           For each num in arr:
               If count == 0:
                   candidate = num
                   count = 1
               Else If num == candidate:
                   count++
               Else:
                   count--
           Return candidate
```

Java Implementation

```
class Solution {
           public int majorityElement(int[] nums) {

              int candidate = -1;
              int count = 0;
```

```
            for (int num : nums) {
                if (count == 0) {
                    candidate = num;
                    count = 1;
                } else if (num == candidate) {
                    count++;
                } else {
                    count--;
                }
            }

            // Verify majority
            count = 0;
            for (int num : nums) {
                if (num == candidate) count++;
            }
            return count > nums.length / 2 ? candidate : -1;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

25. Kadane's Algorithm (Maximum Subarray Sum)

Understanding the Problem NEW

Find the maximum possible sum of any contiguous subarray (a run of consecutive elements) within the array — including the possibility that the best subarray is just a single element, if all numbers are negative.

Brute Force Approach NEW

The naive approach checks every possible subarray's sum (all start/end pairs), which is $O(n^2)$, or $O(n^3)$ if you recompute each subarray's sum from scratch instead of reusing previous work.

Why We Optimize NEW

Kadane's Algorithm asks a smarter local question at each position: **'should I extend the previous subarray by including this element, or is it better to start a fresh subarray here?'** If the running sum so far has gone negative, it can only be dragging future sums down, so it's better to restart from the current element. This is a dynamic-programming insight — the best subarray ending at position i only depends on the best subarray ending at position i-1, not on recomputing everything from scratch — collapsing $O(n^2)$ work into a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the maximum sum of a contiguous subarray.

Visual Diagram

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

       Step 1: current=-2, max=-2
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
        ↑

       Step 2: current=max(1, -2+1)=1, max=1
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
            ↑
```

```
       Step 3: current=max(-3, 1-3)=-2, max=1
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                ↑

       Step 4: current=max(4, -2+4)=4, max=4
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                   ↑

       Step 5: current=max(-1, 4-1)=3, max=4
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                        ↑

       Step 6: current=max(2, 3+2)=5, max=5
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                           ↑

       Step 7: current=max(1, 5+1)=6, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                              ↑

       Step 8: current=max(-5, 6-5)=1, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                                 ↑

       Step 9: current=max(4, 1+4)=5, max=6
       [-2, 1, -3, 4, -1, 2, 1, -5, 4]
                                    ↑

       Result: 6 (subarray [4, -1, 2, 1])
```

Solution Approach

Concept: Dynamic programming - either start new subarray or extend existing.

Algorithm

```
Algorithm KADANE(arr):
           current_max = arr[0]
           global_max = arr[0]
           For i = 1 to n-1:
               current_max = max(arr[i], current_max + arr[i])
               global_max = max(global_max, current_max)
           Return global_max
```

Java Implementation

```
class Solution {
          public int maxSubarraySum(int[] nums) {
              int currentMax = nums[0];
              int globalMax = nums[0];
```

```
            for (int i = 1; i < nums.length; i++) {
                currentMax = Math.max(nums[i], currentMax + nums[i]);
                globalMax = Math.max(globalMax, currentMax);
            }
            return globalMax;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

26. Print Subarray with Maximum Subarray Sum

Understanding the Problem NEW

This is the same problem as Kadane's Algorithm above, except now you also need to report *which* subarray (the actual start and end indices) produced that maximum sum, not just the sum itself.

Brute Force Approach NEW

Without tracking indices, you'd need to re-scan the array afterward trying to find a subarray that matches the maximum sum you calculated — extra, redundant work.

Why We Optimize NEW

The fix is simple: while running Kadane's algorithm, whenever you decide to 'restart' the running sum, also update a `start` marker to the current index; and whenever the running sum beats the best-so-far, save the current `start` and current index as the best `(start, end)` pair. No extra pass is needed — you get the actual subarray boundaries for free, in the same single O(n) sweep.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print the subarray that gives maximum sum.

Visual Diagram

```
Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

       Tracking:
       Start: 0, End: 0, Max: -2
       i=1: curr=1, global=1 → start=1, end=1
       i=2: curr=-2 → no change
       i=3: curr=4, global=4 → start=3, end=3
       i=4: curr=3 → no change
       i=5: curr=5, global=5 → start=3, end=5
       i=6: curr=6, global=6 → start=3, end=6
       i=7: curr=1 → no change
       i=8: curr=5 → no change
```

```
Subarray: indices 3 to 6 → [4, -1, 2, 1]
Sum: 6
```

Solution Approach

Concept: Kadane's algorithm with tracking indices.

Algorithm

```
Algorithm MAX_SUBARRAY_PRINT(arr):
           current_sum = arr[0]
           max_sum = arr[0]
           start = 0, end = 0, temp_start = 0

           For i = 1 to n-1:
               If arr[i] > current_sum + arr[i]:
                   current_sum = arr[i]
                   temp_start = i
               Else:
                   current_sum = current_sum + arr[i]

                 If current_sum > max_sum:
                     max_sum = current_sum
                     start = temp_start
                     end = i

           Return subarray from start to end
```

Java Implementation

```
class Solution {
           public List<Integer> maxSubarrayPrint(int[] nums) {
               int currentSum = nums[0];
               int maxSum = nums[0];
               int start = 0, end = 0, tempStart = 0;

                 for (int i = 1; i < nums.length; i++) {
                     if (nums[i] > currentSum + nums[i]) {
                         currentSum = nums[i];
                         tempStart = i;
                     } else {
                         currentSum = currentSum + nums[i];

                  }

                  if (currentSum > maxSum) {
                      maxSum = currentSum;
                      start = tempStart;
                      end = i;
                  }
```

```
            }

            List<Integer> result = new ArrayList<>();
            for (int i = start; i <= end; i++) {
                result.add(nums[i]);
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

27. Stock Buy and Sell

Understanding the Problem NEW

You're given the price of a stock on each day, and you may buy once and sell once (buy before you sell). Find the maximum profit possible. E.g. prices `[7,1,5,3,6,4]` → buy at 1, sell at 6, profit 5.

Brute Force Approach NEW

The obvious approach: for every pair of buy-day and later sell-day, compute the profit and track the best — that's checking all $O(n^2)$ pairs of days.

Why We Optimize NEW

As you scan through the prices left to right, the best profit if you *sold today* only depends on the **lowest price you've seen so far** (since you must buy before you sell). So you just track the running minimum price seen, and at each day compute `currentPrice - minSoFar`, keeping the best result. This reduces the problem to a single O(n) pass with O(1) space, because you never need to look back at all previous days individually — only the cheapest one matters.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find maximum profit from buying and selling stock once.

Visual Diagram

```
Prices: [7, 1, 5, 3, 6, 4]

       Step 1: min_price=7, max_profit=0
       [7, 1, 5, 3, 6, 4]
        ↑
       min=7, profit=0

       Step 2: 1 < 7 → min=1
       [7, 1, 5, 3, 6, 4]
           ↑
         min=1, profit=0

       Step 3: profit = 5-1=4 → max=4
```

```
       [7, 1, 5, 3, 6, 4]
              ↑
           min=1, profit=4

       Step 4: profit = 3-1=2 → max=4
       [7, 1, 5, 3, 6, 4]
                 ↑

       Step 5: profit = 6-1=5 → max=5
       [7, 1, 5, 3, 6, 4]
                    ↑

       Step 6: profit = 4-1=3 → max=5
       [7, 1, 5, 3, 6, 4]
                       ↑

       Result: 5 (Buy at 1, Sell at 6)
```

Solution Approach

Concept: Track minimum price and calculate profit.

Algorithm

```
Algorithm STOCK_PROFIT(prices):
           min_price = INF
           max_profit = 0
           For each price in prices:
               If price < min_price:
                   min_price = price
               max_profit = max(max_profit, price - min_price)
           Return max_profit
```

Java Implementation

```
class Solution {
           public int maxProfit(int[] prices) {
               int minPrice = Integer.MAX_VALUE;
               int maxProfit = 0;

                 for (int price : prices) {
                     if (price < minPrice) {

                      minPrice = price;
                  }
                     maxProfit = Math.max(maxProfit, price - minPrice);
              }
              return maxProfit;
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

28. Rearrange Array in Alternating Positive/Negative

Understanding the Problem NEW

Rearrange the array so that positive and negative numbers alternate in position, starting with a positive number, while keeping the relative order of positives among themselves and negatives among themselves.

Brute Force Approach NEW

A common first approach is to sort or repeatedly scan-and-swap to interleave elements in place, which can involve a lot of shifting and easily become $O(n^2)$ if done carelessly.

Why We Optimize NEW

Since you need to *preserve relative order*, the clean approach is to first separate all positives into one list and all negatives into another (a single O(n) pass), then interleave them into the result array two-at-a-time. This costs O(n) time, at the price of O(n) extra space for the two temporary lists — a reasonable trade given that maintaining order in-place without extra space is significantly trickier.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rearrange array so positive and negative elements alternate.

Visual Diagram

```
Array: [3, 1, -2, -5, 2, -4]

       Step 1: Separate positives and negatives
       Positives: [3, 1, 2]
       Negatives: [-2, -5, -4]

       Step 2: Place alternately
       Index 0: 3 (positive)
       Index 1: -2 (negative)
       Index 2: 1 (positive)
       Index 3: -5 (negative)
       Index 4: 2 (positive)
       Index 5: -4 (negative)
```

```
       Result: [3, -2, 1, -5, 2, -4]
```

Solution Approach

Concept: Separate arrays for positive and negative.

Algorithm

```
Algorithm REARRANGE_ALTERNATE(arr):
           positives = []
           negatives = []
           For each num in arr:
               If num >= 0: positives.add(num)
               Else: negatives.add(num)

           i=0, j=0, k=0
           While i < positives.size() and j < negatives.size():
               arr[k++] = positives[i++]
               arr[k++] = negatives[j++]
           While i < positives.size():
               arr[k++] = positives[i++]
           While j < negatives.size():
               arr[k++] = negatives[j++]
           Return arr
```

Java Implementation

```
class Solution {
           public void rearrange(int[] arr) {
               List<Integer> positives = new ArrayList<>();
               List<Integer> negatives = new ArrayList<>();

                 for (int num : arr) {
                     if (num >= 0) {
                         positives.add(num);
                     } else {
                         negatives.add(num);
                     }
                 }

                 int i = 0, j = 0, k = 0;

              while (i < positives.size() && j < negatives.size()) {
                  arr[k++] = positives.get(i++);
                  arr[k++] = negatives.get(j++);
              }

              while (i < positives.size()) {
                  arr[k++] = positives.get(i++);
```

```
            }

            while (j < negatives.size()) {
                arr[k++] = negatives.get(j++);
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

29. Next Permutation

Understanding the Problem NEW

Given a permutation (arrangement) of numbers, find the *next* lexicographically greater permutation — the very next arrangement you'd reach if you listed all permutations in sorted order. E.g. `[1,2,3]` → `[1,3,2]`.

Brute Force Approach NEW

The naive approach: generate *all* permutations of the array, sort them, and find the one right after the current arrangement. Generating all permutations costs O(n!) — completely impractical for anything but tiny arrays.

Why We Optimize NEW

You don't need every permutation — you only need to make the **smallest possible increase**. The trick: scan from the right to find the first place where the sequence stops increasing (the 'pivot'); swap that pivot with the smallest element to its right that's still bigger than it; then reverse everything after the pivot's position (since that suffix was in decreasing order, reversing it makes it the smallest possible arrangement). This turns an O(n!) brute enumeration into a clever O(n) sequence of scans, swaps, and a reversal.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the next lexicographically greater permutation.

Visual Diagram

```
Array: [1, 2, 3]

       Step 1: Find first decreasing element from right
       [1, 2, 3]
           ↑
         2 < 3

       Step 2: Find element just greater than 2 from right
       [1, 2, 3]
           ↑ ↑
          2 3
```

```
Step 3: Swap
[1, 3, 2]

Step 4: Reverse suffix after position of 2
[1, 3, 2]

Result: [1, 3, 2]

Example 2:
Array: [3, 2, 1] → Next: [1, 2, 3]

Example 3:
Array: [1, 1, 5] → Next: [1, 5, 1]
```

Solution Approach

Concept: Find pivot, swap with smallest larger element, reverse suffix.

Algorithm

```
Algorithm NEXT_PERMUTATION(arr):
           i = n-2
           While i >= 0 and arr[i] >= arr[i+1]:
               i--

           If i >= 0:
               j = n-1
               While arr[j] <= arr[i]:
                   j--
               Swap arr[i] and arr[j]

           Reverse arr[i+1..n-1]
           Return arr
```

Java Implementation

```
class Solution {
           public void nextPermutation(int[] nums) {
               int i = nums.length - 2;

                 while (i >= 0 && nums[i] >= nums[i+1]) {
                     i--;
                 }

              if (i >= 0) {
                  int j = nums.length - 1;
                  while (nums[j] <= nums[i]) {
                      j--;
                  }
```

```
                swap(nums, i, j);
            }

            reverse(nums, i+1, nums.length - 1);
        }

        private void swap(int[] nums, int i, int j) {
            int temp = nums[i];
            nums[i] = nums[j];
            nums[j] = temp;
        }

        private void reverse(int[] nums, int start, int end) {
            while (start < end) {
                swap(nums, start, end);
                start++;
                end--;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

30. Leaders in an Array

Understanding the Problem NEW

A 'leader' is an element that is greater than every element to its right. Find all the leaders in the array. E.g. in `[16,17,4,3,5,2]`, the leaders are 17, 5, and 2 (each is bigger than everything after it).

Brute Force Approach NEW

The direct approach: for every element, scan all elements to its right and check if it's bigger than all of them — $O(n^2)$ comparisons.

Why We Optimize NEW

Instead of looking forward from each element (which forces rechecking a lot of the same territory), scan **from the right side** and keep a running 'maximum seen so far.' Any element that beats this running maximum is a leader (since everything to its right, by definition, has already been accounted for in that maximum). This collapses the problem into a single right-to-left O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find elements that are greater than all elements to their right.

Visual Diagram

```
Array: [16, 17, 4, 3, 5, 2]

       Traverse from right:
       Step 1: 2 is leader (rightmost)
       [16, 17, 4, 3, 5, 2]
                         ↑
                      leader=2

       Step 2: 5 > 2 → 5 is leader
       [16, 17, 4, 3, 5, 2]
                      ↑
                   leader=5

       Step 3: 3 < 5 → Not leader
```

```
       [16, 17, 4, 3, 5, 2]
                    ↑

       Step 4: 4 < 5 → Not leader
       [16, 17, 4, 3, 5, 2]
                 ↑

       Step 5: 17 > 5 → 17 is leader
       [16, 17, 4, 3, 5, 2]
            ↑
          leader=17

       Step 6: 16 < 17 → Not leader
       [16, 17, 4, 3, 5, 2]
        ↑

       Result: [17, 5, 2]
```

Solution Approach

Concept: Traverse from right, track maximum.

Algorithm

```
Algorithm FIND_LEADERS(arr):
           leaders = []
           max_from_right = -INF

            For i = n-1 down to 0:
                If arr[i] > max_from_right:
                    leaders.add(arr[i])
                    max_from_right = arr[i]

            Reverse leaders
            Return leaders
```

Java Implementation

```
class Solution {
           public List<Integer> findLeaders(int[] arr) {
               List<Integer> leaders = new ArrayList<>();

              int maxFromRight = Integer.MIN_VALUE;

              for (int i = arr.length - 1; i >= 0; i--) {
                  if (arr[i] > maxFromRight) {
                      leaders.add(arr[i]);
                      maxFromRight = arr[i];
                  }
              }
```

```
            Collections.reverse(leaders);
            return leaders;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

31. Longest Consecutive Sequence

Understanding the Problem NEW

Find the length of the longest run of consecutive integers present in the array, regardless of their order or position. E.g. `[100,4,200,1,3,2]` contains the consecutive run 1,2,3,4 — length 4.

Brute Force Approach NEW

A first attempt might sort the array, then scan for the longest consecutive run — that works but costs O(n log n) just for the sort, when the problem can be solved without ordering the whole array.

Why We Optimize NEW

Put every number into a **hash set** for O(1) lookups. Then, for each number, only *start* counting a sequence if that number's predecessor (`num - 1`) is *not* in the set — this guarantees you only ever count each consecutive run starting from its true beginning, instead of re-counting the same run from every one of its members. Each number therefore gets visited a small constant number of times overall, giving true O(n) time — better than sorting, using O(n) extra space for the set.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of longest consecutive elements sequence.

Visual Diagram

```
Array: [100, 4, 200, 1, 3, 2]

       HashSet: {100, 4, 200, 1, 3, 2}

       Check each number:
       100: prev=99 not found → sequence starts here
         length=1
         101 not found → length=1

       4: prev=3 found → skip (start of sequence is 1)
```

```
       200: prev=199 not found → length=1

       1: prev=0 not found → sequence starts here
         len=1, check 2→len=2, check 3→len=3, check 4→len=4, check 5 not found
         max=4

       3: prev=2 found → skip

       2: prev=1 found → skip

       Result: 4 (sequence [1, 2, 3, 4])
```

Solution Approach

Concept: Use HashSet and only start sequence from smallest element.

Algorithm

```
Algorithm LONGEST_CONSECUTIVE(arr):
           set = createSet(arr)
           max_len = 0
           For each num in set:
               If set doesn't contain num-1:
                   current_num = num
                   current_len = 1
                   While set contains current_num+1:
                       current_num++
                       current_len++
                   max_len = max(max_len, current_len)
           Return max_len
```

Java Implementation

```
class Solution {
           public int longestConsecutive(int[] nums) {
               Set<Integer> set = new HashSet<>();
               for (int num : nums) {
                   set.add(num);
               }

                 int maxLen = 0;
                 for (int num : set) {
                     if (!set.contains(num - 1)) {
                         int currentNum = num;

                      int currentLen = 1;

                      while (set.contains(currentNum + 1)) {
                          currentNum++;
                          currentLen++;
```

```
                    }

                    maxLen = Math.max(maxLen, currentLen);
                }
            }
            return maxLen;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

32. Set Matrix Zeros

Understanding the Problem NEW

Given a matrix, if any cell contains a 0, then its entire row and entire column must be set to 0. You need to do this for the whole matrix, ideally without needing a full separate copy of it.

Brute Force Approach NEW

A naive approach: scan the whole matrix, and the moment you see a 0, immediately zero out its row and column right away. The catch is this corrupts the matrix for later checks, causing incorrect over-zeroing. A safer brute fix uses an extra full-size boolean matrix to record which rows/columns need zeroing — this works but costs O(m×n) extra space.

Why We Optimize NEW

Instead of a separate marker matrix, you can reuse the matrix's own **first row and first column** as the marker space (with two extra flags for whether row 0 / column 0 themselves need zeroing, handled separately since they double as markers). You first record markers into the first row/column, then use those markers to zero out the rest of the matrix. This achieves the same O(m×n) time using only O(1) extra space, by cleverly repurposing existing memory instead of allocating new memory.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Set entire row and column to zero if any element is zero.

Visual Diagram

```
Matrix:
       [1, 1, 1]   [1, 0, 1]
       [1, 0, 1] → [0, 0, 0]
       [1, 1, 1]   [1, 0, 1]

       Step 1: Find zero at (1,1)
       [1, 1, 1]
       [1, 0, 1]
```

```
       [1, 1, 1]

       Step 2: Mark row 1 and column 1
       [1, 0, 1]
       [0, 0, 0]
       [1, 0, 1]
```

Solution Approach

Concept: Use first row and column as markers.

Algorithm

```
Algorithm SET_MATRIX_ZERO(matrix):
           m = rows, n = cols
           first_row_zero = false, first_col_zero = false

           Check first row for zero
           Check first column for zero

           For i = 1 to m-1:
               For j = 1 to n-1:
                   If matrix[i][j] == 0:
                       matrix[i][0] = 0
                       matrix[0][j] = 0

           For i = 1 to m-1:
               If matrix[i][0] == 0:
                   Set row i to zero

           For j = 1 to n-1:
               If matrix[0][j] == 0:
                   Set column j to zero

           If first_row_zero: Set first row to zero
           If first_col_zero: Set first column to zero
```

Java Implementation

```
class Solution {
           public void setZeroes(int[][] matrix) {
               int m = matrix.length, n = matrix[0].length;
               boolean firstRowZero = false, firstColZero = false;

                 // Check first row
                 for (int j = 0; j < n; j++) {
                     if (matrix[0][j] == 0) firstRowZero = true;

              }
```

```
            // Check first column
            for (int i = 0; i < m; i++) {
                if (matrix[i][0] == 0) firstColZero = true;
            }

            // Mark zeros
            for (int i = 1; i < m; i++) {
                for (int j = 1; j < n; j++) {
                    if (matrix[i][j] == 0) {
                        matrix[i][0] = 0;
                        matrix[0][j] = 0;
                    }
                }
            }

            // Set rows to zero
            for (int i = 1; i < m; i++) {
                if (matrix[i][0] == 0) {
                    for (int j = 1; j < n; j++) {
                        matrix[i][j] = 0;
                    }
                }
            }

            // Set columns to zero
            for (int j = 1; j < n; j++) {
                if (matrix[0][j] == 0) {
                    for (int i = 1; i < m; i++) {
                        matrix[i][j] = 0;
                    }
                }
            }

            if (firstRowZero) {
                for (int j = 0; j < n; j++) matrix[0][j] = 0;
            }
            if (firstColZero) {
                for (int i = 0; i < m; i++) matrix[i][0] = 0;
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(1)

33. Rotate Matrix by 90 Degrees

Understanding the Problem NEW

Rotate a square matrix 90 degrees clockwise, in-place, without using a second matrix to build the rotated result.

Brute Force Approach NEW

The straightforward approach: create a brand-new matrix, and copy each element from its old position directly to its rotated position using an index formula. This is easy to get right, but costs $O(n^2)$ extra space for the new matrix.

Why We Optimize NEW

You can achieve the same rotation in-place using two simpler operations you may already know: first **transpose** the matrix (swap rows and columns, i.e. flip across the diagonal), then **reverse each row**. Doing these two well-understood $O(n^2)$ operations back-to-back produces exactly a 90-degree clockwise rotation, but using only O(1) extra space, because both transposing and row-reversal can be done by swapping elements within the same matrix.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Rotate a matrix by 90 degrees clockwise.

Visual Diagram

```
Original:           Transpose:           Rotate:
       [1, 2, 3]           [1, 4, 7]           [7, 4, 1]
       [4, 5, 6]      →     [2, 5, 8]      →    [8, 5, 2]
       [7, 8, 9]           [3, 6, 9]           [9, 6, 3]

       Step 1: Transpose
       [1, 4, 7]
       [2, 5, 8]
       [3, 6, 9]

       Step 2: Reverse each row
       [7, 4, 1]
```

```
       [8, 5, 2]
       [9, 6, 3]
```

Solution Approach

Concept: Transpose matrix, then reverse each row.

Algorithm

```
Algorithm ROTATE_90(matrix):
           n = matrix.length

           // Transpose
           For i = 0 to n-1:
               For j = i+1 to n-1:
                   Swap matrix[i][j] and matrix[j][i]

           // Reverse each row
           For i = 0 to n-1:
               For j = 0 to n/2-1:
                   Swap matrix[i][j] and matrix[i][n-1-j]
```

Java Implementation

```
class Solution {
           public void rotate(int[][] matrix) {
               int n = matrix.length;

                 // Transpose
                 for (int i = 0; i < n; i++) {
                     for (int j = i + 1; j < n; j++) {
                         int temp = matrix[i][j];
                         matrix[i][j] = matrix[j][i];
                         matrix[j][i] = temp;
                     }
                 }

                 // Reverse each row
                 for (int i = 0; i < n; i++) {
                     for (int j = 0; j < n/2; j++) {
                         int temp = matrix[i][j];
                         matrix[i][j] = matrix[i][n-1-j];
                         matrix[i][n-1-j] = temp;

                   }
               }
           }
      }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: O(1)

34. Print Matrix in Spiral Manner

Understanding the Problem NEW

Print all elements of a matrix in spiral order — starting from the top-left, going right along the top row, then down the right column, then left along the bottom row, then up the left column, and repeating inward.

Brute Force Approach NEW

Without a clear plan, a beginner might try to track direction changes with ad-hoc conditionals scattered everywhere, which quickly gets tangled and error-prone (easy to double-print or skip a cell) — this isn't really 'brute force vs optimal' so much as 'unstructured vs structured.'

Why We Optimize NEW

The clean approach: maintain four boundaries — `top`, `bottom`, `left`, `right` — representing the current unprinted rectangle. Print along the top row, then shrink `top` down by one; print down the right column, then shrink `right` in by one; and so on for bottom and left, repeating until the boundaries cross. This 'shrinking rectangle' structure guarantees every cell is visited exactly once in the correct order, in a clean O(m×n) pass with O(1) extra space (aside from the output).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print matrix elements in spiral order.

Visual Diagram

```
Matrix:
       [1, 2, 3, 4]
       [5, 6, 7, 8]
       [9, 10, 11, 12]
       [13,14, 15, 16]

       Spiral Order:
       1 → 2 → 3 → 4 ↓
                       8 ↓
                      12 ↓
       16 ← 15 ← 14 ← 13 ↑
```

```
        ↓
        9 → 10 → 11

        Result: [1, 2, 3, 4, 8, 12, 16, 15, 14, 13, 9, 5, 6, 7, 11, 10]
```

Solution Approach

Concept: Traverse in layers (top, right, bottom, left).

Algorithm

```
Algorithm SPIRAL_MATRIX(matrix):
           result = []
           top = 0, bottom = m-1
           left = 0, right = n-1

            While top <= bottom and left <= right:
                // Top row
                For j = left to right:
                    Add matrix[top][j]
                top++

                 // Right column
                 For i = top to bottom:
                     Add matrix[i][right]
                 right--

                 // Bottom row
                 If top <= bottom:
                     For j = right down to left:
                         Add matrix[bottom][j]
                     bottom--

                 // Left column
                 If left <= right:
                     For i = bottom down to top:
                         Add matrix[i][left]
                     left++

             Return result
```

Java Implementation

```
class Solution {

          public List<Integer> spiralOrder(int[][] matrix) {
              List<Integer> result = new ArrayList<>();
              int top = 0, bottom = matrix.length - 1;
              int left = 0, right = matrix[0].length - 1;
```

```
            while (top <= bottom && left <= right) {
                // Top row
                for (int j = left; j <= right; j++) {
                    result.add(matrix[top][j]);
                }
                top++;

                // Right column
                for (int i = top; i <= bottom; i++) {
                    result.add(matrix[i][right]);
                }
                right--;

                // Bottom row
                if (top <= bottom) {
                    for (int j = right; j >= left; j--) {
                        result.add(matrix[bottom][j]);
                    }
                    bottom--;
                }

                // Left column
                if (left <= right) {
                    for (int i = bottom; i >= top; i--) {
                        result.add(matrix[i][left]);
                    }
                    left++;
                }
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m*n) | Space Complexity: O(1)

35. Count Subarrays with Given Sum

Understanding the Problem NEW

Count how many subarrays (contiguous runs) have a sum equal to exactly K. This version assumes the array elements are structured in a way that lets you use prefix sums directly (works for both positive and general integers).

Brute Force Approach NEW

The direct approach checks every possible subarray's sum and counts the matches — O(n²) pairs of start/end indices.

Why We Optimize NEW

As in problem 21, track the running **prefix sum** as you scan left to right, and use a hashmap to remember how many times each prefix sum value has occurred so far. For each new prefix sum `S`, the number of previous positions where the prefix sum was `S - K` tells you exactly how many subarrays ending here sum to K — because the difference between any two prefix sums *is* the sum of the subarray between them. This turns O(n²) pair-checking into a single O(n) pass with a hashmap.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of subarrays with sum equal to K.

Visual Diagram

```
Array: [1, 1, 1], K=2

       Brute Force:
       [1] sum=1 ✗
       [1,1] sum=2 ✓ (1)
       [1,1,1] sum=3 ✗
       [1] sum=1 ✗
       [1,1] sum=2 ✓ (2)
       [1] sum=1 ✗

       Total = 2
```

```
HashMap Method:
prefix=0, map={0:1}

i=0: prefix=1, need=-1 (not found), store 1
i=1: prefix=2, need=0 (found: 1), count=1, store 2
i=2: prefix=3, need=1 (found: 1), count=2, store 3

Result: 2
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm COUNT_SUBARRAYS(arr, K):
           map = {0: 1}
           prefix_sum = 0
           count = 0

           For each num in arr:
               prefix_sum += num
               If map contains (prefix_sum - K):
                   count += map[prefix_sum - K]
               map[prefix_sum] = map.getOrDefault(prefix_sum, 0) + 1

           Return count
```

Java Implementation

```
class Solution {
           public int countSubarraysWithSumK(int[] nums, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, 1);

                 int prefixSum = 0;
                 int count = 0;

                 for (int num : nums) {
                     prefixSum += num;
                     count += map.getOrDefault(prefixSum - K, 0);
                     map.put(prefixSum, map.getOrDefault(prefixSum, 0) + 1);
                 }

              return count;
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

36. Pascal's Triangle

Understanding the Problem NEW

Generate the first n rows of Pascal's Triangle, where each number is the sum of the two numbers directly above it (with edges always being 1). Row 3 (0-indexed) is `[1,3,3,1]`.

Brute Force Approach NEW

A beginner might try to compute each entry directly using the binomial coefficient formula `C(row, col) = row! / (col! × (row-col)!)` from scratch for every cell — this involves repeated, redundant factorial computations and can be slow or prone to overflow for larger rows.

Why We Optimize NEW

Because each entry is defined in terms of the row above it, you can build the triangle **incrementally**: start each row with a 1, and compute every other entry as the sum of the two values above it from the previous row (already computed). This reuses previously computed results instead of recalculating factorials from scratch every time — a simple but classic 'build on what you already know' dynamic-programming pattern, costing O(n²) total for the full triangle, which is optimal since there are O(n²) numbers to produce.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate Pascal's triangle with n rows.

Visual Diagram

```
n=5

       Row 0:                    1
       Row 1:                 1     1
       Row 2:              1     2     1
       Row 3:           1     3     3     1
       Row 4:        1     4     6     4     1

       Construction:
       Row 0: [1]
```

```
       Row 1: [1, 1]
       Row 2: [1, 1+1=2, 1] = [1, 2, 1]
       Row 3: [1, 1+2=3, 2+1=3, 1] = [1, 3, 3, 1]
       Row 4: [1, 1+3=4, 3+3=6, 3+1=4, 1] = [1, 4, 6, 4, 1]
```

Solution Approach

Concept: Each element is sum of two elements above.

Algorithm

```
Algorithm PASCAL_TRIANGLE(numRows):
           result = []
           For row = 0 to numRows-1:
               current = []
               For col = 0 to row:
                   If col == 0 or col == row:
                       current.add(1)
                   Else:
                       prev = result[row-1]
                       current.add(prev[col-1] + prev[col])
               result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> generate(int numRows) {
               List<List<Integer>> result = new ArrayList<>();

                 for (int row = 0; row < numRows; row++) {
                     List<Integer> current = new ArrayList<>();

                       for (int col = 0; col <= row; col++) {
                           if (col == 0 || col == row) {
                               current.add(1);
                           } else {
                               List<Integer> prev = result.get(row - 1);
                               current.add(prev.get(col - 1) + prev.get(col));
                           }
                       }
                       result.add(current);

               }

               return result;
           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: $O(n^2)$

37. Majority Element (n/3 times)

Understanding the Problem NEW

Find all elements that appear **more than n/3 times** in the array. Note there can be at most 2 such elements (since three elements each appearing more than n/3 times would need more than n total elements).

Brute Force Approach NEW

As with the n/2 majority problem, a beginner approach counts occurrences of every distinct value with a hashmap and filters by the n/3 threshold — correct, O(n) time, but O(n) space.

Why We Optimize NEW

The same cancellation idea from Moore's Voting Algorithm can be **extended** to track two candidates and two counters instead of one, since at most two elements can qualify. Elements are paired off against combinations of the two current candidates, with counters incremented/decremented similarly. After one pass, do a second verification pass to confirm the surviving candidates truly occur more than n/3 times (since the voting process can produce false positives without this check). This still runs in O(n) time but drops the extra space down to O(1), using the mathematical guarantee about how many candidates can possibly qualify.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find elements appearing more than n/3 times.

Visual Diagram

```
Array: [3, 2, 3, 1, 3, 4, 2]
       n=7, n/3=2.33

       Boyer-Moore Extended:
       Step 1: candidate1=3, count1=1
       Step 2: 2 != 3 → candidate2=2, count2=1
       Step 3: 3 == candidate1 → count1=2
       Step 4: 1 != both → count1=1, count2=1
```

```
Step 5: 3 == candidate1 → count1=2
Step 6: 4 != both → count1=1, count2=1
Step 7: 2 == candidate2 → count2=2

Candidates: 3 and 2
Verify: 3 appears 3 times, 2 appears 2 times
3 > 2.33 ✓, 2 < 2.33 ✗

Result: [3]
```

Solution Approach

Concept: Extended Boyer-Moore voting for n/3.

Algorithm

```
Algorithm MAJORITY_ELEMENT_N3(arr):
           candidate1 = -1, candidate2 = -1
           count1 = 0, count2 = 0

           For each num in arr:
               If num == candidate1: count1++
               Else If num == candidate2: count2++
               Else If count1 == 0: candidate1 = num; count1 = 1
               Else If count2 == 0: candidate2 = num; count2 = 1
               Else: count1--; count2--

           Verify candidates and return
```

Java Implementation

```
class Solution {
           public List<Integer> majorityElement(int[] nums) {
               int candidate1 = -1, candidate2 = -1;
               int count1 = 0, count2 = 0;

                 for (int num : nums) {
                     if (num == candidate1) {
                         count1++;
                     } else if (num == candidate2) {
                         count2++;
                     } else if (count1 == 0) {
                         candidate1 = num;
                         count1 = 1;
                     } else if (count2 == 0) {
                         candidate2 = num;
                         count2 = 1;

                  } else {
                      count1--;
```

```
                    count2--;
                }
            }

            List<Integer> result = new ArrayList<>();
            count1 = 0; count2 = 0;
            for (int num : nums) {
                if (num == candidate1) count1++;
                else if (num == candidate2) count2++;
            }

            if (count1 > nums.length / 3) result.add(candidate1);
            if (count2 > nums.length / 3) result.add(candidate2);

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

38. 3-Sum Problem

Understanding the Problem NEW

Find all unique triplets of numbers in the array that add up to zero. E.g. in `[-1,0,1,2,-1,-4]`, valid triplets include `[-1,-1,2]` and `[-1,0,1]`.

Brute Force Approach NEW

The most direct approach: three nested loops trying every possible triplet and checking if it sums to zero — O(n³) time, plus extra care needed to avoid counting duplicate triplets.

Why We Optimize NEW

First, **sort** the array (O(n log n)) — this unlocks the two-pointer technique. Then fix one element with a single loop, and for the remaining two, use two pointers moving from both ends of the rest of the array inward: if the sum is too small, move the left pointer right; if too big, move the right pointer left; if it matches, record the triplet and skip over duplicates. This reduces the innermost search from O(n²) (nested loop) to O(n) (two-pointer), giving O(n²) total instead of O(n³) — and sorting also makes duplicate-skipping trivial.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all triplets that sum to zero.

Visual Diagram

```
Array: [-1, 0, 1, 2, -1, -4]
        Sorted: [-4, -1, -1, 0, 1, 2]

        i=0, target=-(-4)=4, left=1, right=5
        -1 + 2 = 1 < 4 → left++
        -1 + 2 = 1 < 4 → left++
        0 + 2 = 2 < 4 → left++
        1 + 2 = 3 < 4 → left++

        i=1, target=-(-1)=1, left=2, right=5
        -1 + 2 = 1 ✓ → [-1, -1, 2]
        -1 + 1 = 0 < 1 → left++ (skip duplicates)
```

```
        i=2, target=-(-1)=1, left=3, right=5
        0 + 2 = 2 > 1 → right--
        0 + 1 = 1 ✓ → [-1, 0, 1]

        Result: [[-1, -1, 2], [-1, 0, 1]]
```

Solution Approach

Concept: Sort array, fix one element, use two-pointer.

Algorithm

```
Algorithm THREE_SUM(arr):
            Sort arr
            result = []
            For i = 0 to n-3:
                If i > 0 and arr[i] == arr[i-1]: Continue
                left = i+1, right = n-1
                While left < right:
                     sum = arr[i] + arr[left] + arr[right]
                     If sum == 0:
                         result.add([arr[i], arr[left], arr[right]])
                         left++; right--
                         Skip duplicates
                     Else If sum < 0:
                         left++
                     Else:
                         right--
            Return result
```

Java Implementation

```
class Solution {
            public List<List<Integer>> threeSum(int[] nums) {
                Arrays.sort(nums);
                List<List<Integer>> result = new ArrayList<>();

                 for (int i = 0; i < nums.length - 2; i++) {
                     if (i > 0 && nums[i] == nums[i-1]) continue;

                     int left = i + 1, right = nums.length - 1;
                     while (left < right) {

                       int sum = nums[i] + nums[left] + nums[right];

                       if (sum == 0) {
                           result.add(Arrays.asList(nums[i], nums[left],
nums[right]));
                           left++;
```

```
                            right--;

                            while (left < right && nums[left] == nums[left-1]) left+
+;
                            while (left < right && nums[right] == nums[right+1])
right--;
                        } else if (sum < 0) {
                            left++;
                        } else {
                            right--;
                        }
                    }
                }

                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(1)

39. 4-Sum Problem

Understanding the Problem NEW

Same idea as 3-Sum, but now find all unique quadruplets (four numbers) that add up to a given target.

Brute Force Approach NEW

The brute approach extends naturally: four nested loops trying every combination — O(n^4) time.

Why We Optimize NEW

Apply the exact same trick as 3-Sum, one level deeper: sort the array, fix the *first two* elements with nested loops, then use the two-pointer technique on the remaining two — turning the innermost O(n^2) search into O(n). This brings total time down from O(n^4) to O(n^3), the same 'fix-and-shrink-with-two-pointers' pattern applied recursively to one more dimension.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all quadruplets that sum to target.

Visual Diagram

```
Array: [1, 0, -1, 0, -2, 2], Target: 0
       Sorted: [-2, -1, 0, 0, 1, 2]

       i=0, j=1: left=2, right=5
       Sum = -2-1+0+2 = -1 < 0 → left++

       Continue searching...
       Found: [-2, -1, 1, 2]
       Found: [-2, 0, 0, 2]
       Found: [-1, 0, 0, 1]

       Result: [[-2, -1, 1, 2], [-2, 0, 0, 2], [-1, 0, 0, 1]]
```

Solution Approach

Concept: Two nested loops + two-pointer technique.

Algorithm

```
Algorithm FOUR_SUM(arr, target):
           Sort arr
           result = []
           For i = 0 to n-4:
               If i > 0 and arr[i] == arr[i-1]: Continue
               For j = i+1 to n-3:
                    If j > i+1 and arr[j] == arr[j-1]: Continue
                    left = j+1, right = n-1
                    While left < right:
                        sum = arr[i] + arr[j] + arr[left] + arr[right]
                        If sum == target:
                            Add quadruplet
                            left++; right--
                            Skip duplicates
                        Else If sum < target:
                            left++
                        Else:
                            right--
           Return result
```

Java Implementation

```
class Solution {
           public List<List<Integer>> fourSum(int[] nums, int target) {
               Arrays.sort(nums);
               List<List<Integer>> result = new ArrayList<>();

                 for (int i = 0; i < nums.length - 3; i++) {
                     if (i > 0 && nums[i] == nums[i-1]) continue;

                    for (int j = i + 1; j < nums.length - 2; j++) {
                        if (j > i + 1 && nums[j] == nums[j-1]) continue;

                         int left = j + 1, right = nums.length - 1;

                         while (left < right) {

                           long sum = (long) nums[i] + nums[j] + nums[left] +
nums[right];

                           if (sum == target) {
                               result.add(Arrays.asList(nums[i], nums[j],
nums[left], nums[right]));
                               left++;
                               right--;
```

```
                                while (left < right && nums[left] == nums[left-1])
left++;
                                while (left < right && nums[right] == nums[right+1])
right--;
                            } else if (sum < target) {
                                left++;
                            } else {
                                right--;
                            }
                        }
                    }
                }

                return result;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n³) | Space Complexity: O(1)

40. Largest Subarray with 0 Sum

Understanding the Problem NEW

Find the length of the longest contiguous subarray whose elements sum to exactly zero (a special case of the 'sum equals K' family of problems, with K=0).

Brute Force Approach NEW

Checking every subarray's sum directly costs $O(n^2)$.

Why We Optimize NEW

Using prefix sums again: if the *same* prefix sum value occurs at two different indices, the subarray between them must sum to zero (since the running total didn't change between those points). Store the **first** occurrence index of each prefix sum in a hashmap; whenever a prefix sum repeats, compute the distance between the current index and that first occurrence to get a candidate length, keeping the maximum. This is a single O(n) pass with a hashmap, reusing exactly the same prefix-sum insight from problems 21, 35, and 40.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the length of largest subarray with sum 0.

Visual Diagram

```
Array: [15, -2, 2, -8, 1, 7, 10, 23]

       Prefix Sum:
       i=0: sum=15, store at index 0
       i=1: sum=13, store at index 1
       i=2: sum=15, found at index 0 → length=2-0=2
       i=3: sum=7, store at index 3
       i=4: sum=8, store at index 4
       i=5: sum=15, found at index 0 → length=5-0=5
       i=6: sum=25, store at index 6
       i=7: sum=48, store at index 7

       Max length = 5 (subarray: [-2, 2, -8, 1, 7])
```

Solution Approach

Concept: Prefix sum with hashmap.

Algorithm

```
Algorithm LARGEST_SUBARRAY_0(arr):
           map = {0: -1}
           sum = 0, max_len = 0
           For i = 0 to n-1:
               sum += arr[i]
               If map contains sum:
                   max_len = max(max_len, i - map[sum])
               Else:
                   map[sum] = i
           Return max_len
```

Java Implementation

```
class Solution {
           public int largestSubarrayWithZeroSum(int[] nums) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, -1);

                 int sum = 0, maxLen = 0;

                 for (int i = 0; i < nums.length; i++) {
                     sum += nums[i];

                     if (map.containsKey(sum)) {
                         maxLen = Math.max(maxLen, i - map.get(sum));
                     } else {
                         map.put(sum, i);
                     }
                 }

                 return maxLen;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

41. Count Number of Subarrays with Given Sum

Understanding the Problem NEW

This is functionally the same question as problem 35 — count subarrays summing to K — just emphasized here for arrays that may include negative numbers, where the earlier sliding-window idea (problem 20) wouldn't be valid.

Brute Force Approach NEW

As before, checking every subarray directly is $O(n^2)$.

Why We Optimize NEW

The same prefix-sum-plus-hashmap technique from problem 35 applies unchanged here, and it works correctly regardless of whether the array has negative numbers — because prefix sums and their differences are well-defined either way, unlike the sliding-window approach which specifically relies on all-positive values to guarantee the window only grows as you add elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count subarrays with sum equal to K (including negative numbers).

Visual Diagram

```
Array: [1, -1, 0, 1], K=0

       HashMap:
       prefix=0, map={0:1}

       i=0: sum=1, need=1, map.get(1)=0, count=0, store 1
       i=1: sum=0, need=0, map.get(0)=1, count=1, store 0
       i=2: sum=0, need=0, map.get(0)=2, count=3, store 0
       i=3: sum=1, need=1, map.get(1)=1, count=4, store 1

       Result: 4 subarrays
       Subarrays: [1,-1], [1,-1,0], [0], [0,1,-1,0]
```

Solution Approach

Concept: Prefix sum with hashmap (same as problem 35).

Algorithm

```
Algorithm COUNT_SUBARRAYS_K(arr, K):
           map = {0: 1}
           sum = 0, count = 0
           For each num in arr:
               sum += num
               count += map.getOrDefault(sum - K, 0)
               map.put(sum, map.getOrDefault(sum, 0) + 1)
           Return count
```

Java Implementation

```
class Solution {
           public int countSubarraysWithSumK(int[] nums, int K) {
               Map<Integer, Integer> map = new HashMap<>();
               map.put(0, 1);

                 int sum = 0, count = 0;

                 for (int num : nums) {
                     sum += num;
                     count += map.getOrDefault(sum - K, 0);
                     map.put(sum, map.getOrDefault(sum, 0) + 1);
                 }

                 return count;
           }
       }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n)

42. Merge Overlapping Subintervals

Understanding the Problem NEW

Given a list of intervals (like meeting times) that may overlap, merge all overlapping ones into a minimal set of non-overlapping intervals. E.g. `[[1,3],[2,6],[8,10]]` → `[[1,6],[8,10]]`.

Brute Force Approach NEW

A naive approach compares every pair of intervals to check for overlap and merges them repeatedly — this can require multiple passes and cost up to $O(n^2)$ or worse, since merging one pair can create new overlaps with other intervals.

Why We Optimize NEW

The trick that unlocks a clean solution: **sort the intervals by start time** first (O(n log n)). Once sorted, you only ever need to compare each interval with the *most recently merged* interval — if it overlaps, extend that merged interval's end; if not, start a new merged interval. This single left-to-right pass after sorting guarantees correctness because overlapping intervals must be adjacent once sorted by start time, avoiding the need to re-compare every pair.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge all overlapping intervals.

Visual Diagram

```
Intervals: [[1,3], [2,6], [8,10], [15,18]]

       Step 1: Sort by start time
       [[1,3], [2,6], [8,10], [15,18]]

       Step 2: Merge
       [1,3] → merge with [2,6] → [1,6]
       [1,6] → next [8,10] doesn't overlap → add [1,6]
       [8,10] → next [15,18] doesn't overlap → add [8,10]
       [15,18] → add [15,18]

       Result: [[1,6], [8,10], [15,18]]
```

```
Visual:
[1,3]   [2,6]     [8,10]    [15,18]
[----] [----]     [----]    [----]
[---------]                 [----]
```

Solution Approach

Concept: Sort intervals and merge overlapping ones.

Algorithm

```
Algorithm MERGE_INTERVALS(intervals):
           Sort intervals by start
           result = []
           current = intervals[0]

           For each interval in intervals[1..n-1]:
               If interval.start <= current.end:
                   current.end = max(current.end, interval.end)
               Else:
                   result.add(current)
                   current = interval
           result.add(current)
           Return result
```

Java Implementation

```
class Solution {
           public int[][] merge(int[][] intervals) {
               Arrays.sort(intervals, (a, b) -> a[0] - b[0]);

                 List<int[]> result = new ArrayList<>();
                 int[] current = intervals[0];
                 result.add(current);

                 for (int[] interval : intervals) {
                     if (interval[0] <= current[1]) {
                         current[1] = Math.max(current[1], interval[1]);
                     } else {
                         current = interval;
                         result.add(current);
                     }

               }

               return result.toArray(new int[result.size()][]);
           }
       }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

43. Merge Two Sorted Arrays Without Extra Space

Understanding the Problem NEW

You have two already-sorted arrays and need to merge them into one fully sorted sequence — but with the constraint that you cannot use any extra array; everything must be rearranged within the two given arrays' existing memory.

Brute Force Approach NEW

The typical merge (like in Merge Sort) uses a temporary array to hold the merged result before copying it back — simple and O(n+m) time, but it violates the 'no extra space' constraint here.

Why We Optimize NEW

The **gap method** (inspired by Shell Sort) solves this in-place: start with a gap roughly half the combined length, and compare-and-swap elements that are 'gap' positions apart across both arrays (treating them conceptually as one combined sequence); shrink the gap each round (dividing by 2, rounding up) until the gap reaches 1. This is a clever reuse of an old sorting idea (Shell Sort's gap sequence) to solve a merging problem without extra memory, trading a bit of extra time complexity (O((n+m) log(n+m))) for the O(1) space guarantee.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Merge two sorted arrays without using extra space.

Visual Diagram

```
arr1: [1, 3, 5, 7]
       arr2: [2, 4, 6, 8]

       Step 1: Compare arr1[3]=7 with arr2[0]=2
       7 > 2 → swap → arr1=[1,3,5,2], arr2=[7,4,6,8]

       Step 2: Sort arr2 → arr2=[4,6,7,8]
       arr1=[1,3,5,2] → arr2=[4,6,7,8]

       Step 3: arr1[2]=5 with arr2[0]=4
       5 > 4 → swap → arr1=[1,3,4,2], arr2=[5,6,7,8]
```

```
Step 4: Sort arr2 → arr2=[5,6,7,8]
arr1=[1,3,4,2] → arr2=[5,6,7,8]

Step 5: arr1[1]=3 with arr2[0]=5 → 3 < 5 (continue)

After full merging:
arr1 = [1,2,3,4], arr2 = [5,6,7,8]
```

Solution Approach

Concept: Gap method (Shell sort).

Algorithm

```
Algorithm MERGE_WITHOUT_SPACE(arr1, arr2):
           n = len(arr1), m = len(arr2)
           gap = ceil((n+m)/2)

           While gap > 0:
               i = 0
               While i + gap < n+m:
                   If i < n and i+gap < n:
                       Compare arr1[i] and arr1[i+gap]
                   Else If i < n and i+gap >= n:
                       Compare arr1[i] and arr2[i+gap-n]
                   Else:
                       Compare arr2[i-n] and arr2[i+gap-n]
                   i++
               gap = gap <= 1 ? 0 : (gap+1)/2
```

Java Implementation

```
class Solution {
           public void merge(int[] arr1, int[] arr2) {
               int n = arr1.length, m = arr2.length;
               int gap = (n + m + 1) / 2;

                 while (gap > 0) {
                     int i = 0, j = gap;

                    while (j < n + m) {
                        if (j < n) {
                            if (arr1[i] > arr1[j]) {

                                swap(arr1, i, j);
                            }
                      } else if (i < n) {
                           if (arr1[i] > arr2[j - n]) {
                               int temp = arr1[i];
```

```
                            arr1[i] = arr2[j - n];
                            arr2[j - n] = temp;
                        }
                    } else {
                        if (arr2[i - n] > arr2[j - n]) {
                            swap(arr2, i - n, j - n);
                        }
                    }
                    i++;
                    j++;
                }

                gap = gap <= 1 ? 0 : (gap + 1) / 2;
            }
        }

        private void swap(int[] arr, int i, int j) {
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
```

Time & Space Complexity

Time Complexity: O((n+m) log(n+m)) | Space Complexity: O(1)

44. Find Repeating and Missing Number

Understanding the Problem NEW

An array of size n should contain every number from 1 to n exactly once, but due to a data error, one number is duplicated (appears twice) and another is missing entirely. Find both numbers.

Brute Force Approach NEW

A beginner approach: use a hashmap or frequency array to count occurrences of every number, then scan through 1..n to find which count is 2 (the repeating number) and which is 0 (the missing one) — correct, O(n) time, but O(n) extra space.

Why We Optimize NEW

You can solve this with pure arithmetic and O(1) space: let `S` and `S2` be the actual sum and sum-of-squares of the array, and `S_expected` / `S2_expected` be what they'd be for a perfect 1..n array. The differences `(S - S_expected)` and `(S2 - S2_expected)` give you two equations in two unknowns (the missing and repeating numbers), which you can solve directly. (An XOR-based method achieves the same result while avoiding overflow issues with large sums of squares.) Either way, you go from O(n) space down to O(1) by using number-theoretic relationships instead of explicit counting.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

In array of size n with numbers 1 to n, find the one repeating and one missing number.

Visual Diagram

```
Array: [3, 1, 2, 5, 3], n=5
       Expected: [1, 2, 3, 4, 5]
       Actual:   [3, 1, 2, 5, 3]

       Method 1: Sum and Square Sum
       Expected Sum = 15, Actual Sum = 14
       Expected Square Sum = 55, Actual Square Sum = 48
```

```
Let x = missing, y = repeating
y - x = 14-15 = -1 ... (1)
y² - x² = 48-55 = -7 ... (2)

From (1): y = x - 1
Substitute in (2): (x-1)² - x² = -7
x² - 2x + 1 - x² = -7
-2x + 1 = -7
-2x = -8
x = 4 (missing)
y = 3 (repeating)

Method 2: XOR
XOR all numbers: 3^1^2^5^3 = 4
XOR 1..5: 1^2^3^4^5 = 1
XOR result = 4^1 = 5

Result: Missing=4, Repeating=3
```

Solution Approach

Concept: Use XOR or sum methods.

Algorithm (XOR Method)

Algorithm REPEATING_MISSING(arr):

xor_all = 0

For i in arr: xor_all ^= i

For i = 1 to n: xor_all ^= i

// xor_all = missing ^ repeating

Find rightmost set bit of xor_all

xor1 = 0, xor2 = 0

For each num in arr:

If num has set bit: xor1 ^= num

Else: xor2 ^= num

For i = 1 to n:

If i has set bit: xor1 ^= i

Else: xor2 ^= i

Check which is missing and which is repeating

Java Implementation

```
class Solution {

          public int[] findErrorNums(int[] nums) {
              int n = nums.length;
              int xor = 0;

              // XOR all numbers in array and 1..n
              for (int i = 0; i < n; i++) {
                  xor ^= nums[i];
                  xor ^= (i + 1);
              }
              // xor = missing ^ repeating

              // Find rightmost set bit
              int setBit = xor & -xor;

              int xor1 = 0, xor2 = 0;

              // Partition numbers
              for (int num : nums) {
                  if ((num & setBit) != 0) {
                      xor1 ^= num;
                  } else {
                      xor2 ^= num;
                  }
              }

              for (int i = 1; i <= n; i++) {
                  if ((i & setBit) != 0) {
                      xor1 ^= i;
                  } else {
                      xor2 ^= i;
                  }
              }

              // Find which is repeating
              for (int num : nums) {
                  if (num == xor1) {
                      return new int[]{xor1, xor2};
                  }
              }
              return new int[]{xor2, xor1};
          }
      }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

45. Count Inversions

Understanding the Problem NEW

Count the number of 'inversions' in the array — pairs of indices (i, j) where i comes before j but arr[i] > arr[j]. This is essentially a measure of 'how far the array is from being sorted.'

Brute Force Approach NEW

The direct approach checks every pair (i, j) and counts how many are inverted — $O(n^2)$ comparisons, which is fine for small arrays but too slow for large ones.

Why We Optimize NEW

This is a beautiful reuse of Merge Sort's structure: while merging two already-sorted halves, if an element from the right half gets placed *before* an element still remaining in the left half, that single placement actually represents *multiple* inversions at once — specifically, one inversion for every remaining element in the left half (since the whole left half comes before that position and is bigger than the current right-half element). By counting these during the merge step instead of comparing every pair separately, you piggyback inversion-counting onto Merge Sort's existing O(n log n) structure, avoiding the $O(n^2)$ brute pairwise check entirely.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count the number of inversions (i < j and arr[i] > arr[j]).

Visual Diagram

```
Array: [2, 4, 1, 3, 5]

        Brute Force:
        (2,1), (4,1), (4,3) = 3 inversions

        Merge Sort Method:
        [2, 4, 1, 3, 5]
        Divide: [2, 4] [1, 3, 5]
          [2] [4] [1, 3, 5]
            [1, 3] [5]
```

```
              [1] [3]

        Merge and count:
        [2,4] and [1,3,5]:
        2 > 1 → inversion +2 (2,1 and 4,1)
        2 < 3 → continue
        4 > 3 → inversion +1 (4,3)
        Total inversions = 3

        Result: 3
```

Solution Approach

Concept: Modified merge sort.

Algorithm

```
Algorithm COUNT_INVERSIONS(arr):
            Return mergeSort(arr, 0, n-1)

        Algorithm mergeSort(arr, low, high):
            If low >= high: Return 0
            mid = (low+high)/2
            inv = mergeSort(arr, low, mid)
            inv += mergeSort(arr, mid+1, high)
            inv += merge(arr, low, mid, high)
            Return inv

        Algorithm merge(arr, low, mid, high):
            temp = [], inv = 0
            i=low, j=mid+1
            While i <= mid and j <= high:
                If arr[i] <= arr[j]:
                    temp.add(arr[i++])
                Else:
                    temp.add(arr[j++])
                    inv += (mid - i + 1)
            Add remaining elements
            Copy temp back to arr
            Return inv
```

Java Implementation

```
class Solution {
            public int countInversions(int[] arr) {

                return mergeSort(arr, 0, arr.length - 1);
            }

            private int mergeSort(int[] arr, int low, int high) {
```

```
        if (low >= high) return 0;

        int mid = (low + high) / 2;
        int inv = mergeSort(arr, low, mid);
        inv += mergeSort(arr, mid + 1, high);
        inv += merge(arr, low, mid, high);

        return inv;
    }

    private int merge(int[] arr, int low, int mid, int high) {
        int[] temp = new int[high - low + 1];
        int i = low, j = mid + 1, k = 0;
        int inv = 0;

        while (i <= mid && j <= high) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
                inv += (mid - i + 1);
            }
        }

        while (i <= mid) temp[k++] = arr[i++];
        while (j <= high) temp[k++] = arr[j++];

        System.arraycopy(temp, 0, arr, low, temp.length);
        return inv;
    }
}
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

46. Reverse Pairs

Understanding the Problem NEW

A variant of counting inversions: count pairs (i, j) with i < j where arr[i] > 2 × arr[j] — a stricter 'more than double' condition instead of simply 'bigger.'

Brute Force Approach NEW

Checking every pair directly costs $O(n^2)$, same as basic inversion counting.

Why We Optimize NEW

The same modified-merge-sort trick from Count Inversions applies here, with one twist: because the condition involves a factor of 2 rather than a simple >, you can't count matches with the same single merge pointer used for the normal merge step — you need a **separate pointer** that specifically walks ahead checking the '> 2×' condition, before doing the standard merge comparison. Counting these matches during the merge (rather than via nested loops) again keeps the total time at O(n log n) instead of $O(n^2)$, reusing the divide-and-conquer structure but adapting the counting condition to the specific inequality being asked about.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count pairs where i < j and arr[i] > 2*arr[j].

Visual Diagram

```
Array: [1, 3, 2, 3, 1]

        Brute Force:
        i=0: 1 > 2*1=2? No
        i=1: 3 > 2*2=4? No, 3 > 2*3=6? No, 3 > 2*1=2? Yes → 1 pair
        i=2: 2 > 2*3=6? No, 2 > 2*1=2? No
        i=3: 3 > 2*1=2? Yes → 1 pair
        Total: 2 pairs

        Modified Merge Sort:
        [1, 3, 2, 3, 1]
        Count pairs during merge: (3,1), (3,1) = 2 pairs
```

```
        Result: 2
```

Solution Approach

Concept: Modified merge sort with two-pointer counting.

Algorithm

```
Algorithm REVERSE_PAIRS(arr):
            Return mergeSort(arr, 0, n-1)

        Algorithm mergeSort(arr, low, high):
            If low >= high: Return 0
            mid = (low+high)/2
            count = mergeSort(arr, low, mid)
            count += mergeSort(arr, mid+1, high)
            count += merge(arr, low, mid, high)
            Return count

        Algorithm merge(arr, low, mid, high):
            count = 0
            j = mid+1

            // Count reverse pairs
            For i = low to mid:
                While j <= high and arr[i] > 2*arr[j]:
                    j++
                count += (j - (mid+1))

            // Normal merge
            Merge two halves
            Return count
```

Java Implementation

```
class Solution {
            public int reversePairs(int[] nums) {
                return mergeSort(nums, 0, nums.length - 1);
            }

            private int mergeSort(int[] nums, int low, int high) {
                if (low >= high) return 0;

               int mid = (low + high) / 2;
               int count = mergeSort(nums, low, mid);
               count += mergeSort(nums, mid + 1, high);
               count += merge(nums, low, mid, high);

               return count;
```

```
        }

        private int merge(int[] nums, int low, int mid, int high) {
            int count = 0;
            int j = mid + 1;

            for (int i = low; i <= mid; i++) {
                while (j <= high && (long) nums[i] > 2L * nums[j]) {
                    j++;
                }
                count += (j - (mid + 1));
            }

            // Normal merge
            int[] temp = new int[high - low + 1];
            int i = low;
            j = mid + 1;
            int k = 0;

            while (i <= mid && j <= high) {
                if (nums[i] <= nums[j]) {
                    temp[k++] = nums[i++];
                } else {
                    temp[k++] = nums[j++];
                }
            }

            while (i <= mid) temp[k++] = nums[i++];
            while (j <= high) temp[k++] = nums[j++];

            System.arraycopy(temp, 0, nums, low, temp.length);
            return count;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n log n) | Space Complexity: O(n)

47. Maximum Product Subarray

Understanding the Problem NEW

Find the contiguous subarray whose product (not sum) of elements is the largest. This is trickier than the max-sum version because multiplying by a negative number can flip a very small (very negative) product into the largest one, and multiplying by zero wipes everything out.

Brute Force Approach NEW

The naive approach checks every subarray's product directly — $O(n^2)$, and it's easy to make sign-handling mistakes even then.

Why We Optimize NEW

A single running-maximum (like in Kadane's Algorithm) isn't enough here, because a very negative running product can become the *best* possible product if it gets multiplied by another negative number. The fix: track **both** a running maximum *and* a running minimum product ending at each position. When you hit a negative number, swap max and min (since multiplying flips their roles), then update both using the current element. This dual-tracking adapts Kadane's single-value dynamic-programming idea to handle the sign-flipping behaviour of multiplication, still in a single O(n) pass.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find the subarray with maximum product.

Visual Diagram

```
Array: [2, 3, -2, 4]

      All subarrays:
      [2] = 2
      [2,3] = 6
      [2,3,-2] = -12
      [2,3,-2,4] = -48
      [3] = 3
      [3,-2] = -6
      [3,-2,4] = -24
```

```
      [-2] = -2
      [-2,4] = -8
      [4] = 4

      Maximum = 6 (subarray [2,3])

      Array: [-2, 0, -1]
      Maximum = 0

      Array: [-2, 3, -4]
      All subarrays:
      [-2] = -2
      [-2,3] = -6
      [-2,3,-4] = 24
      [3] = 3
      [3,-4] = -12
      [-4] = -4
      Maximum = 24
```

Solution Approach

Concept: Track max and min products.

Algorithm

```
Algorithm MAX_PRODUCT_SUBARRAY(arr):
          max_product = arr[0]
          min_product = arr[0]
          result = arr[0]

          For i = 1 to n-1:
              If arr[i] < 0:
                  Swap max_product and min_product

                 max_product = max(arr[i], max_product * arr[i])
                 min_product = min(arr[i], min_product * arr[i])

                 result = max(result, max_product)

          Return result
```

Java Implementation

```
class Solution {
          public int maxProduct(int[] nums) {

              int maxProduct = nums[0];
              int minProduct = nums[0];
              int result = nums[0];
```

```
            for (int i = 1; i < nums.length; i++) {
                if (nums[i] < 0) {
                    int temp = maxProduct;
                    maxProduct = minProduct;
                    minProduct = temp;
                }

                maxProduct = Math.max(nums[i], maxProduct * nums[i]);
                minProduct = Math.min(nums[i], minProduct * nums[i]);

                result = Math.max(result, maxProduct);
            }

            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(1)

Summary Table

Problem	Time	Space	Key Technique
Largest Element	O(n)	O(1)	Simple iteration
Second Largest	O(n)	O(1)	Two variables
Check Sorted	O(n)	O(1)	Compare adjacent
Remove Duplicates	O(n)	O(1)	Two-pointer
Left Rotate by One	O(n)	O(1)	Store and shift
Left Rotate by D	O(n)	O(1)	Reversal algorithm
Move Zeros	O(n)	O(1)	Two-pointer
Linear Search	O(n)	O(1)	Traversal
Union	O(n+m)	O(n+m)	Two-pointer
Missing Number	O(n)	O(1)	Sum/XOR
Max Consecutive Ones	O(n)	O(1)	Counting
Single Number	O(n)	O(1)	XOR
Longest Subarray K (Pos)	O(n)	O(1)	Sliding window
Longest Subarray K	O(n)	O(n)	Prefix sum
Two Sum	O(n)	O(n)	HashMap
Sort 0,1,2	O(n)	O(1)	DNF Algorithm
Majority Element	O(n)	O(1)	Moore's Voting
Kadane's	O(n)	O(1)	DP
Stock Buy Sell	O(n)	O(1)	Greedy
Rearrange Alt	O(n)	O(n)	Separation
Next Permutation	O(n)	O(1)	Finding pivot
Leaders	O(n)	O(n)	Right traversal
Longest Consecutive	O(n)	O(n)	HashSet
Set Matrix Zero	O(m*n)	O(1)	Marking
Rotate Matrix	$O(n^2)$	O(1)	Transpose + Reverse
Spiral Matrix	O(m*n)	O(1)	Layer traversal
Count Subarrays	O(n)	O(n)	Prefix sum
Pascal's Triangle	$O(n^2)$	$O(n^2)$	DP

Majority n/3	O(n)	O(1)	Extended Voting
3-Sum	$O(n^2)$	O(1)	Two-pointer
4-Sum	$O(n^3)$	O(1)	Two-pointer
Largest 0 Sum	O(n)	O(n)	Prefix sum
Merge Intervals	O(n log n)	O(n)	Sorting
Merge Arrays	O((n+m)log(n+m))	O(1)	Gap method
Repeating Missing	O(n)	O(1)	XOR
Count Inversions	O(n log n)	O(n)	Merge Sort
Reverse Pairs	O(n log n)	O(n)	Merge Sort
Max Product	O(n)	O(1)	DP

— End of enhanced Chapter 2: Arrays Problems — remaining chapters continue in the full edition —